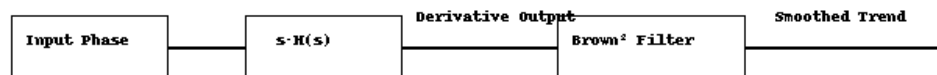


Type equation here. PLL Derivative and Brown Filter Analysis

This document illustrates the structure and frequency responses for a second-order PLL system's derivative output, with and without a second-order Brown low-pass filter applied.

Block Diagram

The diagram below shows the signal flow and where each transfer function is measured:



- The input phase on the picture is the PLL input signal.
- The pre-integrator outp

ut is modeled by $s \cdot H(s)$ where $H(s)$ is defined as:

Here is the accurate frequency response of the second-order system using the Laplace-domain transfer function:

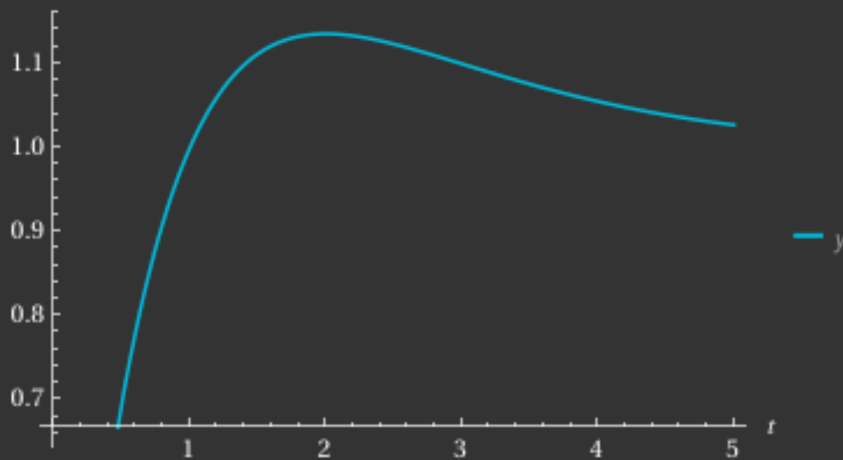
$$H(s) = \frac{2\zeta\omega_n s + \omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

The unit response for $z=1$, $\omega_n = 1$ is:

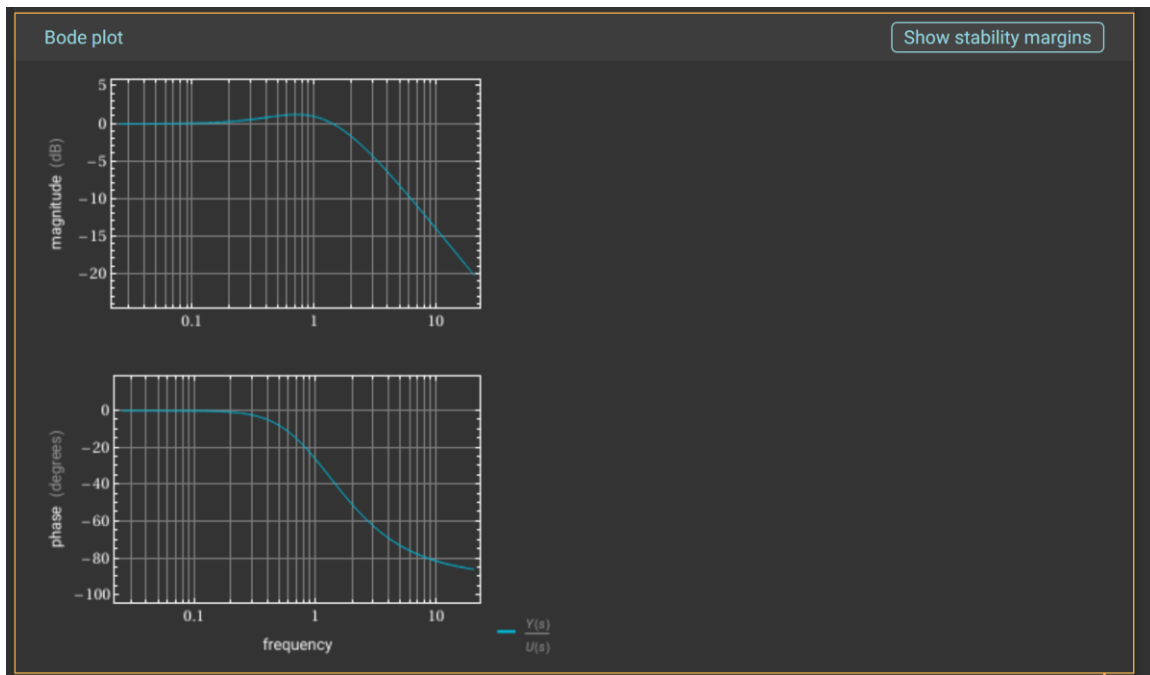
systems model

transfer function $\frac{2s + 1}{s^2 + 2s + 1}$

Unit step response plot



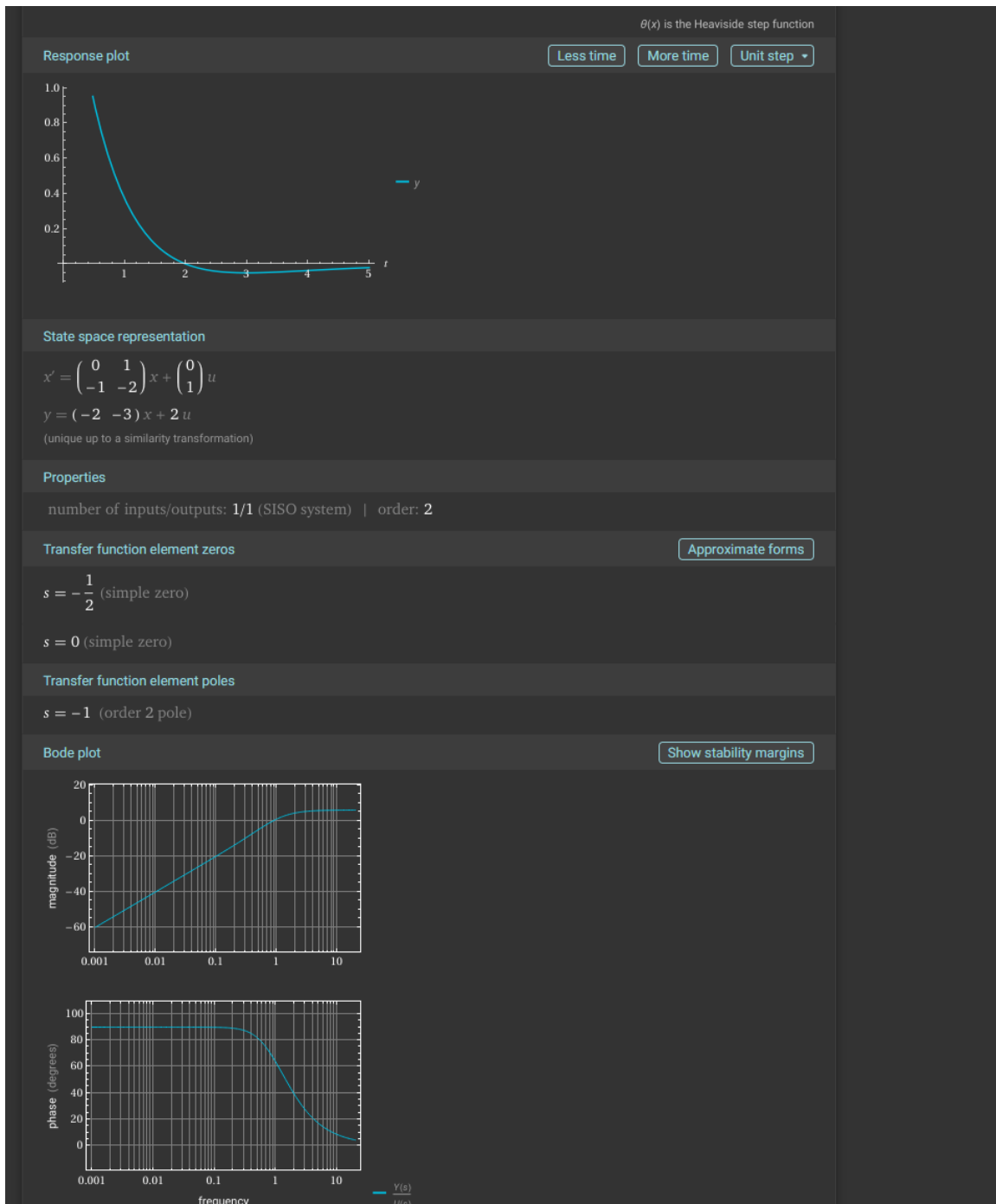
And the frequency response and phase plot are:



- This output is optionally passed through a second-order Brown filter to produce a smoothed trend signal.

Bode Plot

The plot below compares the frequency responses of the derivative output (red) and the filtered version (blue).



If the output derivative is filtered with a second order Brown low pass filter the system full response become a 4 order system with the following characteristic:

systems model

transfer function

$$\left(s \times \frac{2s+1}{s^2+2s+1} \right) \times \frac{1}{(1+s)^2}$$

response to $\theta(t)$

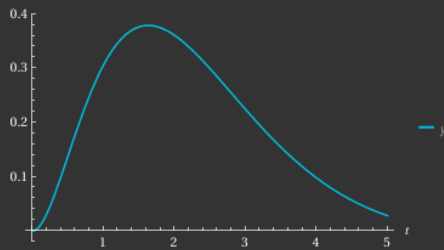
$\theta(x)$ is the Heaviside step function

Response plot

Less time

More time

Unit step ▾



State space representation

$$\dot{x} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -1 & -4 & -6 & -4 \end{pmatrix} x + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} u$$

$$y = (0 \ 1 \ 2 \ 0) x + 0 u$$

(unique up to a similarity transformation)

Properties

number of inputs/outputs: 1/1 (SISO system) | order: 4

Transfer function element zeros

Approximate forms

$$s = -\frac{1}{2} \text{ (simple zero)}$$

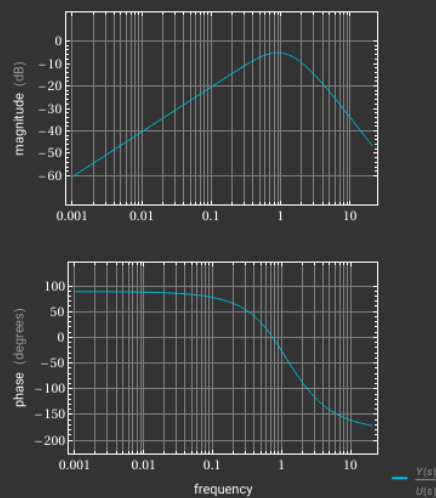
$$s = 0 \text{ (simple zero)}$$

Transfer function element poles

$$s = -1 \text{ (order 4 pole)}$$

Bode plot

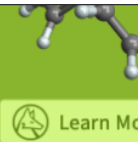
Show stability margins



Nyquist plot

Show Nyquist grid

Show stability margins



systems model

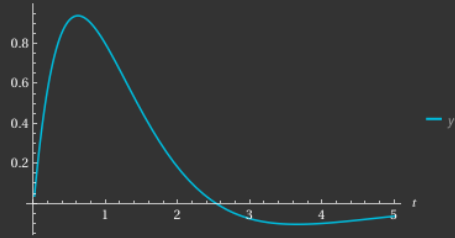
$$\text{transfer function } \frac{s(2s+1)}{s^2+2s+1} \left(-\frac{2}{1+s} - \frac{1}{(1+s)^2} \right)$$

Unit step response plot

Less time

More time

A Plain Text



State space representation

$$\dot{x} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ -1 & -5 & -10 & -10 & -5 \end{pmatrix} x + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} u$$

$$y = (0 \ 1 \ 5 \ 8 \ 4) x + 0 u$$

(unique up to a similarity transformation)

Properties

number of inputs/outputs: **1/1** (SISO system) | order: **5**

Transfer function element zeros

Approximate forms

$s = -1$ (simple zero)

$s = -\frac{1}{2}$ (order 2 zero)

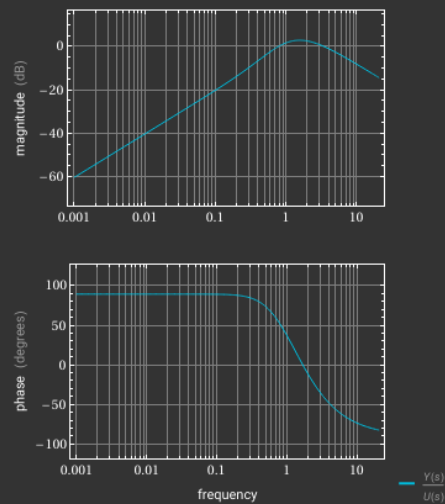
$s = 0$ (simple zero)

Transfer function element poles

$s = -1$ (order 4 pole)

Bode plot

Show stability margins

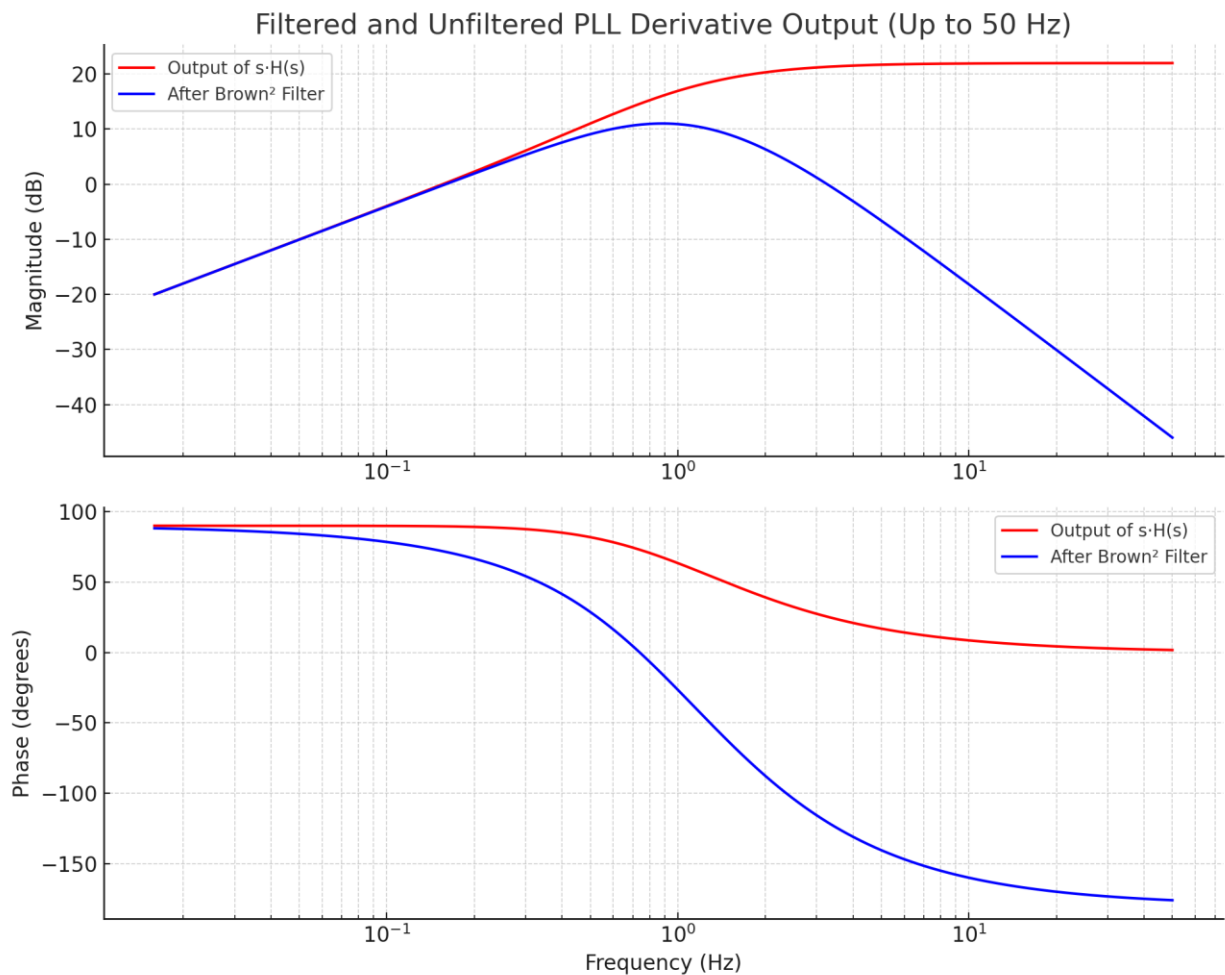


Nyquist plot

Show Nyquist grid

Show stability margins

Im



- The raw derivative ($s \cdot H(s)$) shows significant high-frequency amplification.
- The Brown² filter reduces high-frequency noise while preserving trend responsiveness near the PLL's natural frequency.

Title: Relationship Between Alpha and Cutoff Frequency in First-Order Exponential Filters

- 1. Introduction** A first-order exponential filter (also called a one-pole IIR low-pass filter or Brown's exponential smoother) is commonly used in digital signal processing for its simplicity and computational efficiency. The key design parameter, alpha (α), controls the responsiveness or smoothing level of the filter. Understanding the relationship between α and the filter's 3 dB cutoff

frequency is important for both filter design In the frequency domain the IIR first order filter is defined as:

$$H(s) = \frac{1}{\tau s + 1}$$

2. Filter Definition The filter is defined by the recursive equation:

$$y[n] = (1 - \alpha) \cdot y[n - 1] + \alpha \cdot x[n]$$

Where:

- $x[n]$: input signal
- $y[n]$: output signal
- $\alpha \in (0,1)$: smoothing parameter

3. Frequency Response and Cutoff Definition The frequency response in the Z-domain is:

$$H(e^{j\omega}) = \frac{\alpha}{1 - (1 - \alpha)e^{-j\omega}}$$

Taking the squared magnitude gives:

$$|H(e^{j\omega})|^2 = \frac{\alpha^2}{1 - 2(1 - \alpha)\cos(\omega) + (1 - \alpha)^2}$$

The cutoff frequency ω_c is defined as the frequency where the gain is reduced by 3 dB:

$$|H(e^{j\omega_c})|^2 = \frac{1}{2}$$

Solving this gives the **exact expression**:

$$\omega_c = \arccos\left(\frac{\alpha^2 + 2\alpha - 2}{2\alpha - 2}\right)$$

4. Approximate Formula A commonly used approximation based on continuous-time filter equivalence via the bilinear transform is:

$$\alpha \approx 1 - e^{-2\pi f_c / f_s}$$

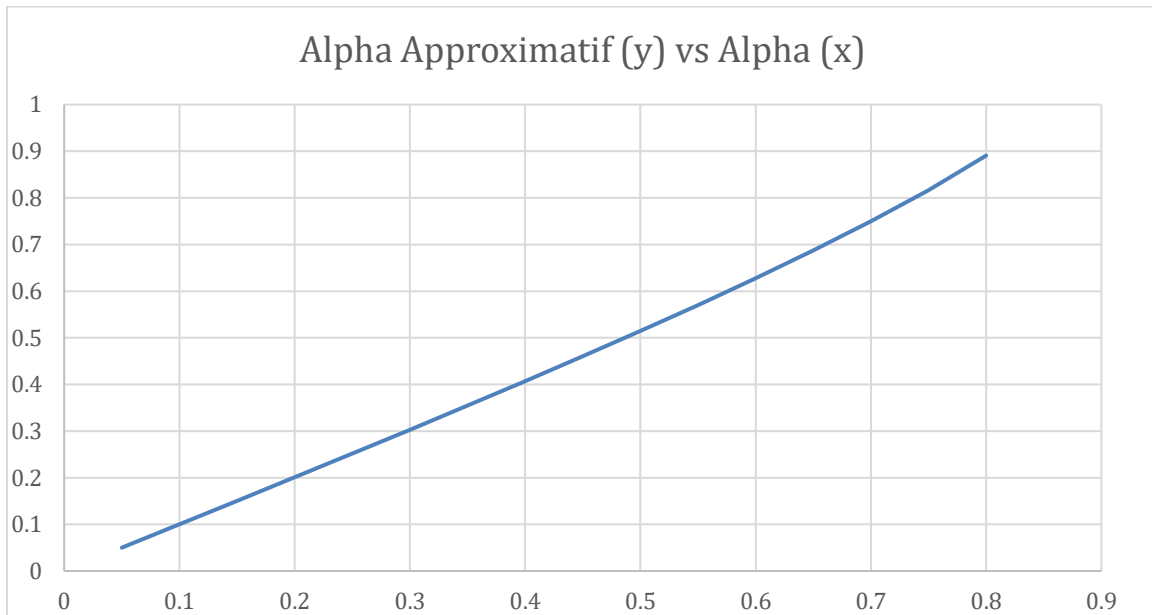
Where:

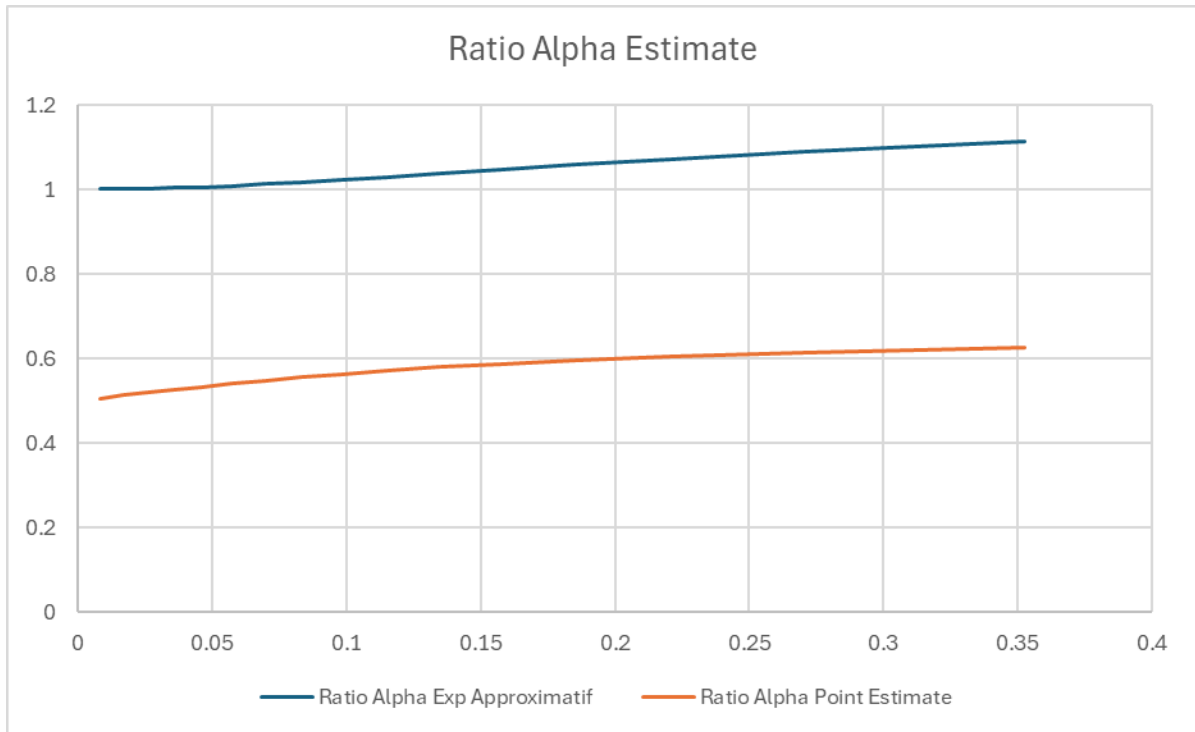
- α

- f_c : desired cutoff frequency in Hz
- f_s : sampling rate in Hz

Inverting this gives:

$$f_c \approx -\frac{f_s}{2\pi} \ln(1 - \alpha)$$





7. Time to Reach 90% of Final Value (Step Response) When a unit step is applied to a first-order exponential filter, the output follows:

$$y[n] = 1 - (1 - \alpha)^n$$

To determine how many samples $n_{90\%}$ it takes for the response to reach 90% of its final value:

$$1 - (1 - \alpha)^n = 0.9 \Rightarrow (1 - \alpha)^n = 0.1$$

Solving for n:

$$n_{90\%} = \frac{\ln(0.1)}{\ln(1 - \alpha)} \approx \frac{-2.3026}{\ln(1 - \alpha)}$$

Examples:

α Samples to 90% ($n_{90\%}$)

0.01 230

0.05 45

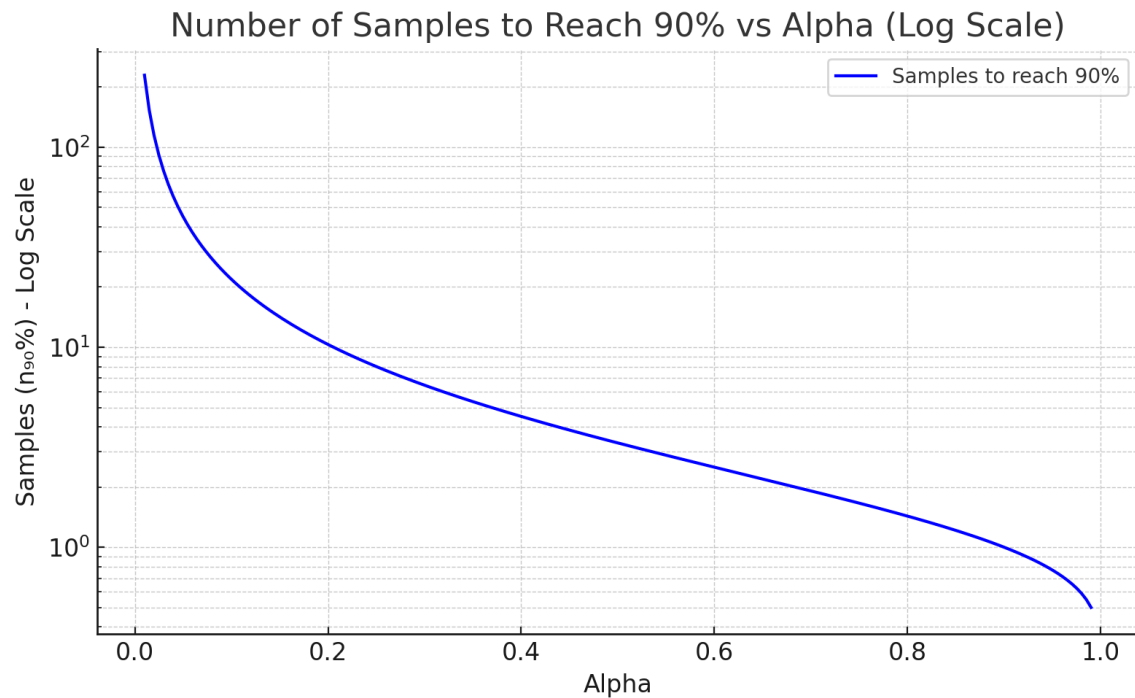
0.1 22

0.3 8

α Samples to 90% ($n_{90\%}$)

0.5 4.3

0.9 1.05



5. When to Use Each Exact Expression:

- Use when precise control of cutoff is required
- Recommended for analytical work or design specifications where exact cutoff mapping is critical

Approximation:

- Sufficient for most practical filter implementations
- Easier to compute, especially when $f_c \ll f_s$
- Useful when deriving α from a desired analog cutoff frequency

6. Conclusion The relationship between α and the 3 dB cutoff frequency is fundamental to the effective use of exponential smoothing filters. Both the exact and approximate

expressions have their place, depending on the required precision and computational constraints.

Appendix: Common Approximate Values

α **Approx. ω_c (*rad/sample*)**

0.01 0.063

0.1 0.628

0.5 1.81

0.9 2.81

0.99 3.08

$$\omega_{ENB}/\omega_c$$

5. Equivalent Noise Bandwidth (ENBW) The Equivalent Noise Bandwidth (ENBW), also called ω_{NEB} , represents the bandwidth of an ideal rectangular filter that passes the same noise power as the actual filter. For the exponential filter:

$$\omega_{ENB} = \frac{\alpha}{2 - \alpha} \cdot \pi$$

$$f_{ENB} = \frac{\alpha}{2(2 - \alpha)}$$

This is derived by integrating the squared frequency response over the Nyquist interval.

6. Comparison Between ω_c and ω_{NEB}

- Both are bandwidth measures, but serve different purposes:
 - ω_c : frequency at which the gain drops to -3 dB
 - ω_{NEB} : area-equivalent bandwidth for noise
- For small α , both increase approximately linearly with α
- $\omega_{NEB} < \omega_c$ for small α , but they converge for $\alpha \rightarrow 1$ to 1

Appendix: Common Approximate Values

α Approx. ω_c (rad/sample)

0.01 0.063

0.1 0.628

0.5 1.81

0.9 2.81

0.99 3.08

Frequency Response of Brown's Double Exponential Smoothing Filter

Brown's Double Exponential Smoothing filter is a two-stage filter designed to capture both the level and the trend in a time series. Unlike a simple exponential filter (single smoothing), it applies exponential smoothing twice, enabling it to adapt to linear trends more effectively.

Filter Definition

The filter operates in two stages:

- First smoothing: $S_1(t) = \alpha x(t) + (1 - \alpha) S_1(t-1)$
- Second smoothing: $S_2(t) = \alpha S_1(t) + (1 - \alpha) S_2(t-1)$

The forecast is computed using: $\hat{x}(t+1) = 2 S_1(t) - S_2(t)$

Transfer Function (Z-domain)

The Z-domain transfer function of the second smoother is:

$$H(z) = \alpha^2 / (1 - (1 - \alpha)z^{-1})^2$$

This is equivalent to a second-order IIR low-pass filter.

Frequency Response

To understand the filter's behavior in the frequency domain, we evaluate the transfer function on the unit circle:

$$z = e^{j\omega}, \omega \in [0, \pi]$$

The magnitude response becomes:

$$|H(e^{j\omega})| = \alpha^2 / [1 - 2(1 - \alpha)\cos(\omega) + (1 - \alpha)^2]$$

This formula shows that low frequencies ($\omega \approx 0$) pass through with near-unit gain, while high frequencies ($\omega \approx \pi$) are increasingly attenuated. The smaller the α , the stronger the smoothing and the steeper the attenuation.

Slope and Prediction Insight

The difference between the first and second smoothing stages, $S_1(t) - S_2(t)$, represents the recent trend or slope. This difference is scaled by $\alpha / (1 - \alpha)$ to estimate the slope, allowing the filter to make a one-step forecast that incorporates both the level and trend of the signal:

$$Trend \approx (\alpha / (1 - \alpha)) * (S^1 - S^2)$$

This trend-following behavior is particularly useful in forecasting applications where the underlying data is expected to follow a consistent linear trajectory in the short term.

Figure: Frequency response (in dB) of Brown's double exponential filter for various α values.

Relation Between Laplace and Z Transform in Exponential Filters

This document explains how the Laplace-domain representation of an exponential filter relates to its discrete-time Z-transform equivalent, particularly in the context of Brown's exponential smoothing.

Continuous-Time (Laplace Domain)

A first-order low-pass filter in the Laplace domain is represented as:

$$H(s) = 1/(\tau s + 1)$$

Where:

- τ is the time constant,
- s is the Laplace transform variable.

Discrete-Time (Z Domain)

The equivalent Z-domain representation of a single exponential smoother is:

$$H(z) = \alpha / [1 - (1 - \alpha)z^{-1}]$$

Where:

- $\alpha \in [0, 1]$ is the smoothing factor,
- $(1 - \alpha) = \beta$ is the decay factor (memory).

Mapping Between Domains

The approximate transformation from Laplace to Z-domain using backward Euler is:

$$s \approx (1 - z^{-1}) / T$$

Substituting into the Laplace transfer function yields:

$$H(z) \approx 1 / [(\tau/T)(1 - z^{-1}) + 1]$$

Letting $\alpha = T / (\tau + T)$, this simplifies to:

$$H(z) = \alpha / [1 - (1 - \alpha)z^{-1}]$$

Parameter Equivalence

To relate α and τ directly:

$$\alpha = T/(\tau + T)$$

$$\tau = T * (1 - \alpha)/\alpha$$

Thus, larger τ means slower response and smaller α (heavier smoothing).

Smaller τ gives faster response and larger α (less smoothing).

Interpretation of β

The term $\beta = 1 - \alpha$ represents the filter's memory. A larger β means the filter gives more weight to older samples, resulting in stronger smoothing.

Example: Frequency Response

The graph below shows the frequency response of the Brown double exponential filter in dB for several α values. It demonstrates that smaller α values result in stronger attenuation of high-frequency components.

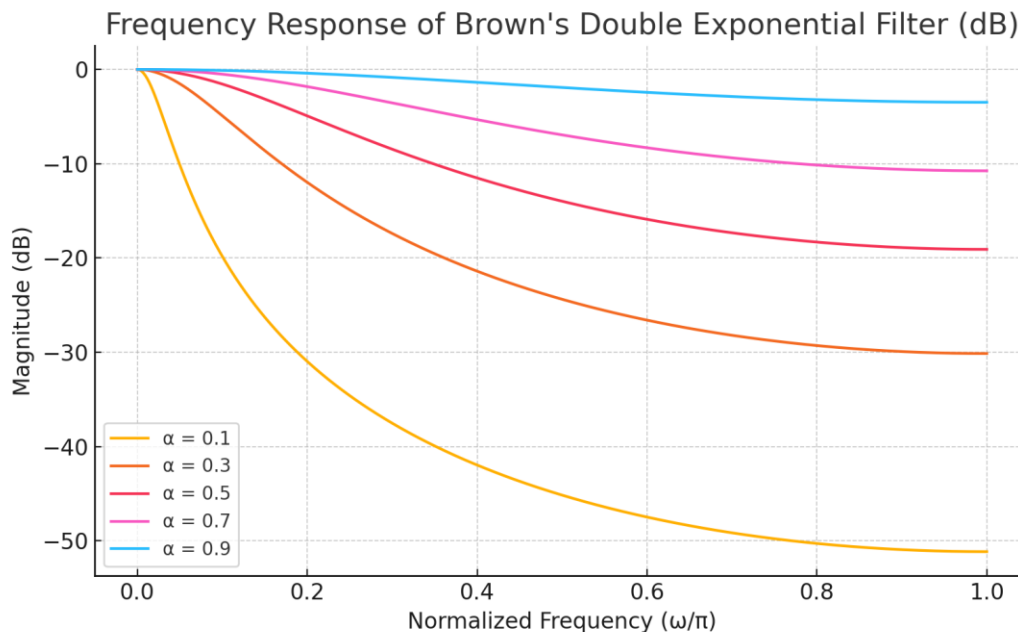


Figure: Frequency response (in dB) of Brown's double exponential filter for various α values.

Generated by ChatGPT · OpenAI

Brown Filter Response for the trend measurement:

Input interpretation

systems model

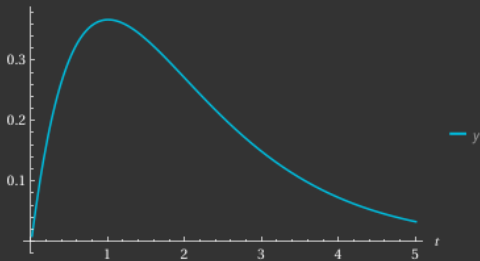
$$\text{transfer function } \frac{1}{1+s} - \frac{1}{(1+s)^2}$$

Unit step response plot

Less time

More time

Unit step ▾



State space representation

$$\dot{x} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & -3 & -3 \end{pmatrix} x + \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} u$$

$$y = (0 \ 1 \ 1) x + 0 u$$

(unique up to a similarity transformation)

Properties

number of inputs/outputs: **1/1** (SISO system) | order: **3**

Transfer function element zeros

$s = -1$ (simple zero)

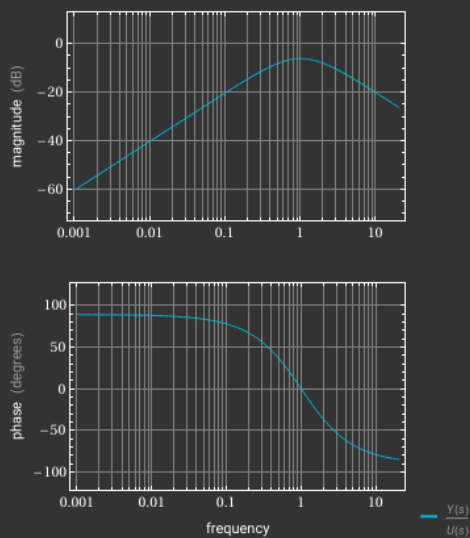
$s = 0$ (simple zero)

Transfer function element poles

$s = -1$ (order 2 pole)

Bode plot

Show stability margins



Nyquist plot

Show Nyquist grid

Show stability margins

Im

Re

Brown Filter Response for the Amplitude measurement:

Input interpretation

systems model

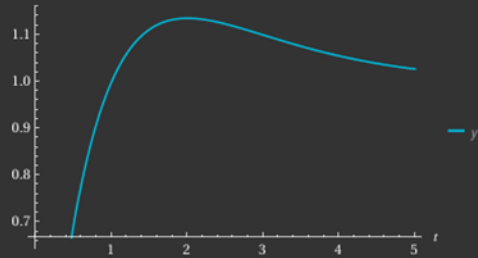
$$\text{transfer function } \frac{1}{1+s} + \frac{1}{1+s} - \frac{1}{(1+s)^2}$$

Unit step response plot

Less time

More time

Unit step ▾



State space representation

$$\dot{x} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & -3 & -3 \end{pmatrix} x + \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} u$$

$$y = (1 \ 3 \ 2) x + 0 u$$

(unique up to a similarity transformation)

Properties

number of inputs/outputs: 1/1 (SISO system) | order: 3

Transfer function element zeros

Approximate forms

$s = -1$ (simple zero)

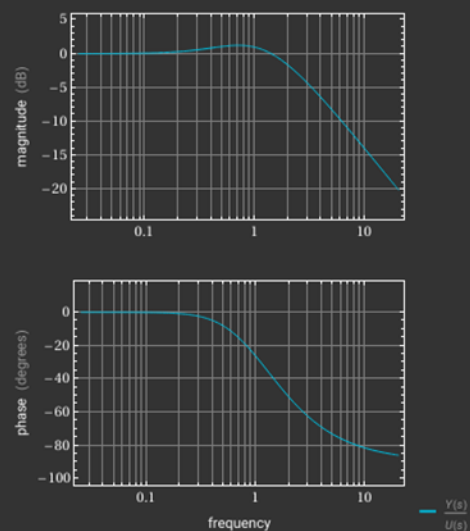
$s = -\frac{1}{2}$ (simple zero)

Transfer function element poles

$s = -1$ (order 2 pole)

Bode plot

Show stability margins



Nyquist plot

Show Nyquist grid

Show stability margins



Brown Filter Response for the Amplitude with no trend component:

input interpretation

systems model

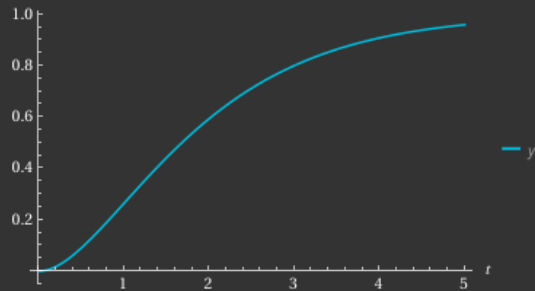
transfer function $\frac{1}{(1+s)^2}$

Unit step response plot

Less time

More time

Unit step ▾



State space representation

$$\dot{x} = \begin{pmatrix} 0 & 1 \\ -1 & -2 \end{pmatrix} x + \begin{pmatrix} 0 \\ 1 \end{pmatrix} u$$

$$y = \begin{pmatrix} 1 & 0 \end{pmatrix} x + 0 u$$

(unique up to a similarity transformation)

Properties

number of inputs/outputs: 1/1 (SISO system) | order: 2

Transfer function element zeros

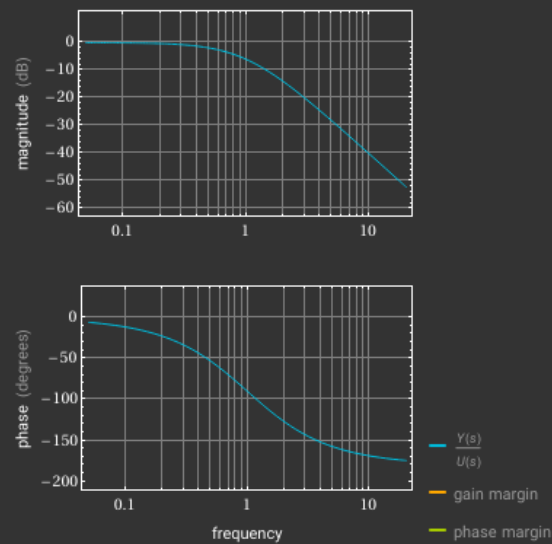
(none)

Transfer function element poles

$s = -1$ (order 2 pole)

Bode plot

Hide stability margins



Nyquist plot

Show Nyquist grid

Show stability margins

Equivalent model for a derivative Brown Filter Response:

Input interpretation

systems model

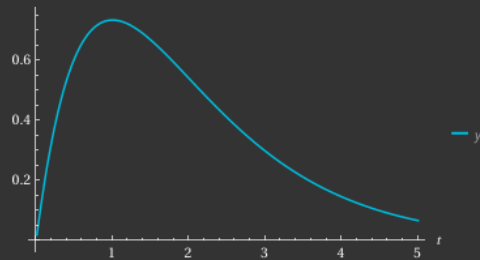
transfer function $\frac{2s + 1}{s^2 + 2s + 1} - \left(\frac{1}{1 + s}\right)^2$

Unit step response plot

Less time

More time

Unit step ▾



State space representation

$$\dot{x} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -1 & -4 & -6 & -4 \end{pmatrix} x + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} u$$

$$y = (0 \ 2 \ 4 \ 2)x + 0u$$

(unique up to a similarity transformation)

Properties

number of inputs/outputs: 1/1 (SISO system) | order: 4

Transfer function element zeros

$s = -1$ (order 2 zero)

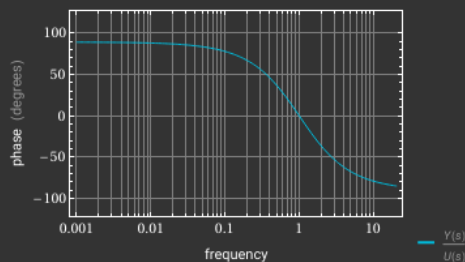
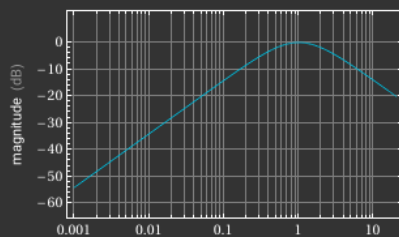
$s = 0$ (simple zero)

Transfer function element poles

$s = -1$ (order 2 pole)

Bode plot

Show stability margins



Nyquist plot

Show Nyquist grid

Show stability margins

systems model

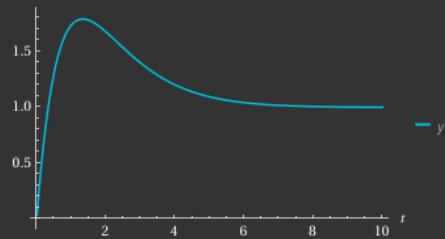
$$\text{transfer function } 2 \times \frac{2s + 1}{s^2 + 2s + 1} - \left(\frac{1}{1 + s} \right)^2$$

Unit step response plot

Less time

More time

Unit step ▾



State space representation

$$\dot{x} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -1 & -4 & -6 & -4 \end{pmatrix} x + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} u$$

$$y = (1 \ 6 \ 9 \ 4) x + 0 u$$

(unique up to a similarity transformation)

Properties

number of inputs/outputs: 1/1 (SISO system) | order: 4

Transfer function element zeros

Approximate forms

$s = -1$ (order 2 zero)

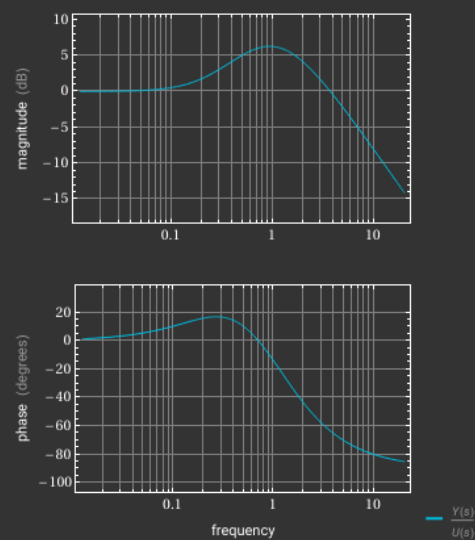
$s = -\frac{1}{4}$ (simple zero)

Transfer function element poles

$s = -1$ (order 2 pole)

Bode plot

Show stability margins



Nyquist plot

Show Nyquist grid

Show stability margins

Im

Input interpretation

systems model

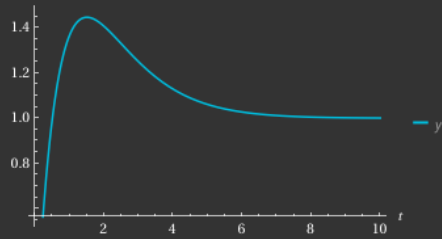
transfer function $2 \times \frac{2s + 1}{s^2 + 2s + 1} - \frac{1}{1 + s}$

Unit step response plot

Less time

More time

Unit step ▾



State space representation

$$\dot{x} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & -3 & -3 \end{pmatrix} x + \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} u$$

$$y = (1 \ 4 \ 3) x + 0 u$$

(unique up to a similarity transformation)

Properties

number of inputs/outputs: 1/1 (SISO system) | order: 3

Transfer function element zeros

Approximate forms

$s = -1$ (simple zero)

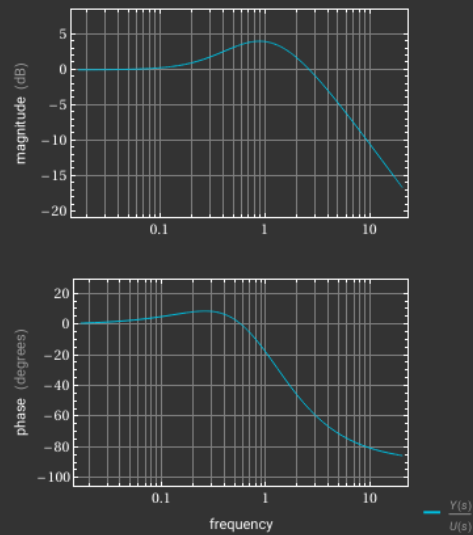
$s = -\frac{1}{3}$ (simple zero)

Transfer function element poles

$s = -1$ (order 2 pole)

Bode plot

Show stability margins



Nyquist plot

Show Nyquist grid

Show stability margins

Im

Input interpretation

systems model

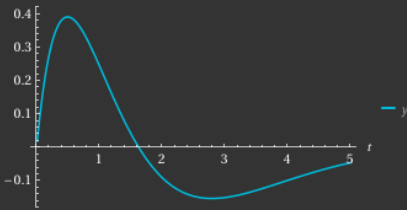
transfer function $\frac{2s + 1}{s^2 + 2s + 1} - \left(\frac{2s + 1}{s^2 + 2s + 1} \right)^2$

Unit step response plot

Less time

More time

Unit step ▾



State space representation

$$\dot{x} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ -1 & -6 & -15 & -20 & -15 & -6 \end{pmatrix} x + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} u$$

$$y = (0 \ 0 \ 1 \ 4 \ 5 \ 2)x + 0 \ u$$

(unique up to a similarity transformation)

Properties

number of inputs/outputs: 1/1 (SISO system) | order: 6

Transfer function element zeros

Approximate forms

$s = -1$ (order 2 zero)

$s = -\frac{1}{2}$ (simple zero)

A Plain Text

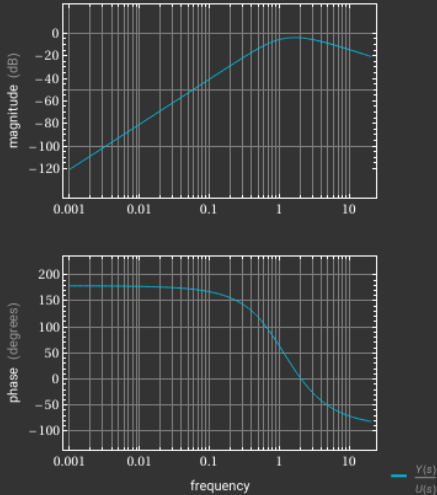
$s = 0$ (order 2 zero)

Transfer function element poles

$s = -1$ (order 4 pole)

Bode plot

Show stability margins



Nyquist plot

Show Nyquist grid

Show stability margins

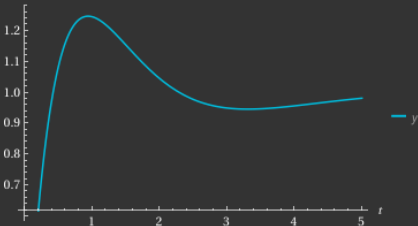
Input interpretation

systems model

transfer function $2 \times \frac{2s + 1}{s^2 + 2s + 1} - \left(\frac{2s + 1}{s^2 + 2s + 1} \right)^2$

Unit step response plot

Less time More time Unit step ▾



State space representation

$$\dot{x} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ -1 & -6 & -15 & -20 & -15 & -6 \end{pmatrix} x + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} u$$

$$y = (1 \ 6 \ 15 \ 20 \ 14 \ 4) x + 0 u$$

(unique up to a similarity transformation)

Properties

number of inputs/outputs: 1/1 (SISO system) | order: 6

Transfer function element zeros

Approximate forms

$s = -1$ (order 2 zero)

$s = -\frac{1}{2}$ (simple zero)

$s = -\frac{1}{2} - \frac{i}{2}$ (simple zero)

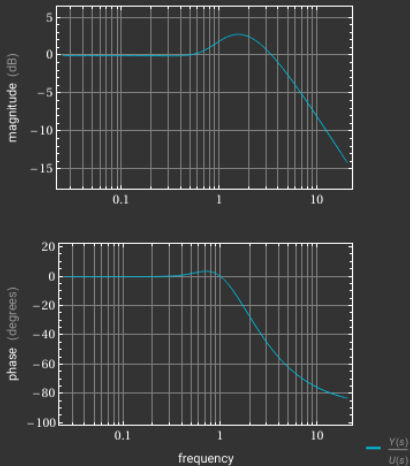
$s = -\frac{1}{2} + \frac{i}{2}$ (simple zero)

Transfer function element poles

$s = -1$ (order 4 pole)

Bode plot

Show stability margins



Nyquist plot

Show Nyquist grid Show stability margins

Frequency Response of PLL Equivalent digital Filter

What is the PLL Digital Frequency?

First let define a signal $x(t)$ with frequency f_0 signal period T_0 . The

digital sequence $x[k]$ is sampled at discrete times $t_k = kT_s$, where T_s is the sampling period.

Now the signal can be written as:

$$x[k] = A \cos(\omega t_k + \varphi) = A \cos(\omega k T_s + \varphi)$$

If we define a new parameters called the digital frequency $\Omega = \omega T_s$ we can now write the above equation as:

$$x[k] = A \cos((\omega T_s)k + \varphi) = A \cos(\Omega k + \varphi)$$

In other word, we are simply normalizing the frequency of the physical signal to how fast it was sampled. Note also that the digital frequency Ω is the angular frequency ω (rad/s) times the sampling period T_s (s/sample), so the units of digital frequency are rad/sample.

As a concept one major reason for using Ω instead of ω is that signal processing techniques fundamentally take in a sequence of numbers $x[n]$ and operate on them in the same way regardless of the sampling period. By generalizing the sequences to the parameters k the time become an integer index and we can generalize signal processing techniques from that.

Going back to the equation of a second order ADPLL as presented in

All digital phase-locked loop: concepts, design and applications

Y.R. Shayan
T. Le-Ngoc

We have the following transfert fonction et parameters defined as:

$$H(S) = \frac{2\eta\omega_n S + \omega_n^2}{S^2 + 2\eta\omega_n S + \omega_n^2} \quad (2)$$

The values of C_1 and C_2 can be found when the sampling frequency f_s is high compared to the natural frequency f_n of the loop ($\omega_n T \ll 1$) and

$$\begin{aligned} C_2 &= 2\eta\omega_n T \\ C_1 &= C_2^2/4\eta^2 \end{aligned} \quad (3)$$

where

$$T = \frac{1}{f_s}, \quad \omega_n = 2\pi f_n$$

and η is the damping factor.

Another aspect to consider for a ADPLL is that there is three frequencies of concern for system performance consideration. We have the resonance frequency ω_n , which appear directly as a parameters in the transfer function $H(s)$, the 3 dB bandwidth ω_c and the noise bandwidth equivalent ω_{nb} related to the measure in the energy of the noise at the output relative to a perfect windows type bandwidth this parameter (NEB) can be called B_n but it is generally better to define it as ω_{nb} as to distinguish it from the parameter ω_n . The relation for B_n or ω_{nb} can be shown to be dependent of ω_n and the damping factor ζ as:

$$B_n = \frac{\omega_n}{2} \left(\zeta + \frac{1}{4\zeta} \right)$$

Or

$$\omega_{nb} = \frac{\omega_n}{2} \left(\zeta + \frac{1}{4\zeta} \right)$$

The reference from the equation above is R. Best, Phase Locked Loops - Design, Simulation and Applications (6th Edition), McGraw Hill, 2007

Also the the 3dB bandwidth can be approximated with this formula extracts from a visual graphic interpretation from ChatGpt:

3 dB Bandwidth vs. Damping in Second-Order ADPLL

This document presents an improved model for estimating the 3 dB bandwidth of a second-order analog or digital phase-locked loop (ADPLL) system as a function of the damping factor ζ .

Measured Bandwidth Data

Measured 3 dB bandwidths (normalized by natural frequency f_n):

Damping Factor (ζ)	3 dB Bandwidth / f_n
0.70	2.04
1.00	2.46
1.25	2.86
1.50	3.28

Improved Analytical Fit

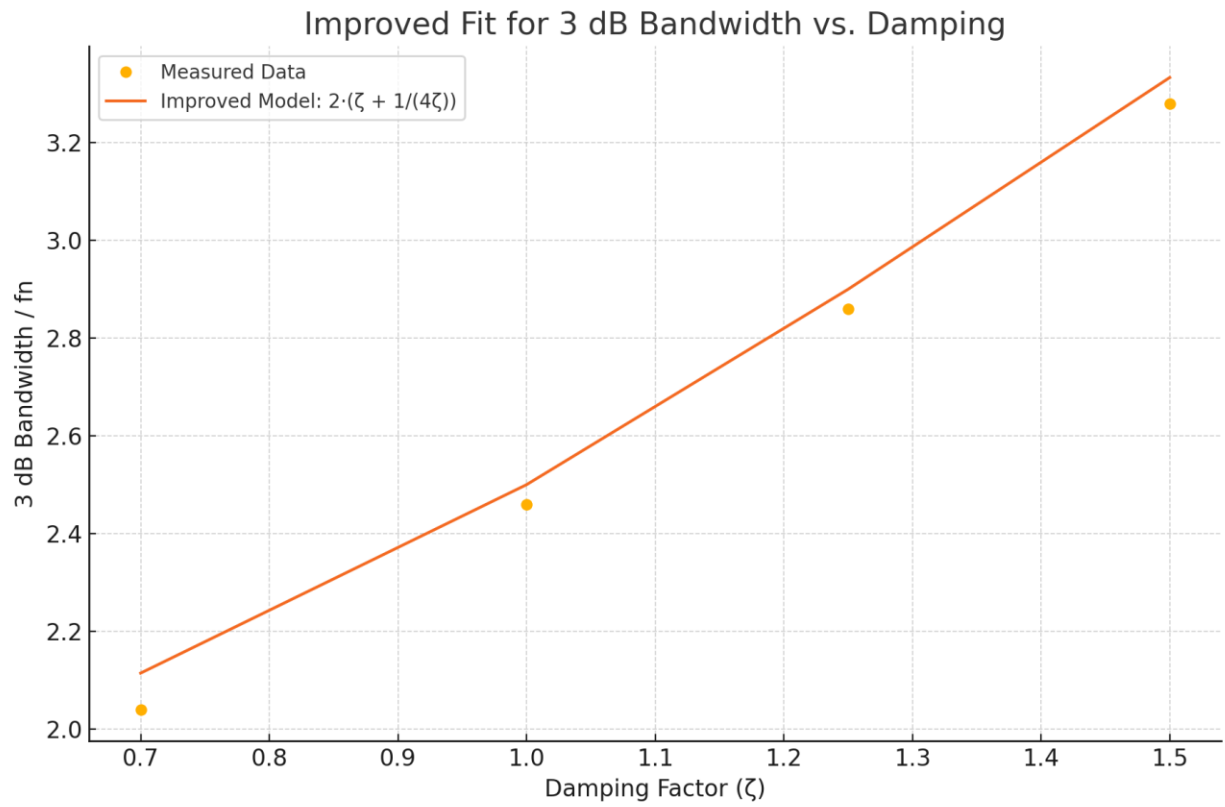
Rather than relying on a linear approximation, the following analytical model provides a better match:

$$f_c \approx 2f_n \left(\zeta + \frac{1}{4\zeta} \right)$$

This expression reflects the natural shape of the transfer function and improves accuracy across a wider range of damping factors compared to simple linear regression.

Comparison Graph

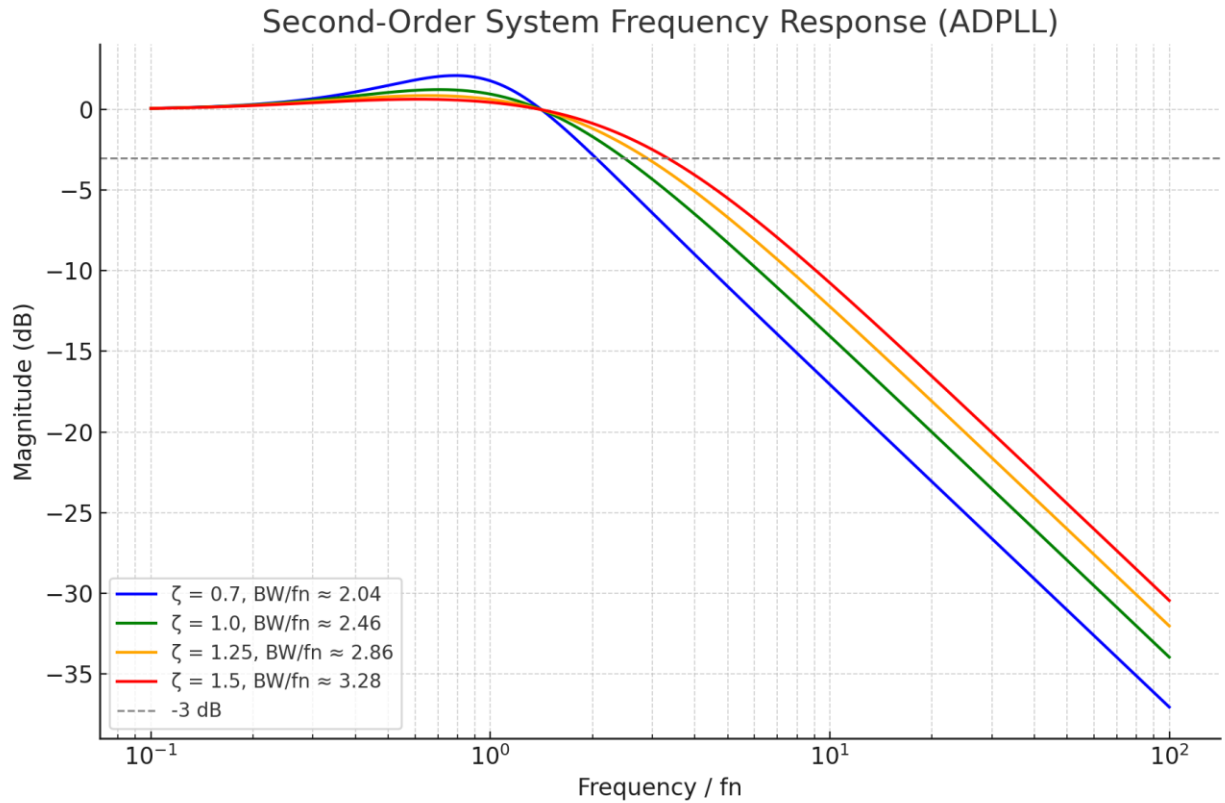
The following graph compares the measured 3 dB bandwidths to values predicted by the improved formula:



Generated by ChatGPT · OpenAI

Frequency Response Curves

This figure shows the frequency response of the second-order ADPLL system for different damping factors ζ . The 3 dB points are visually confirmed in each case.



Estimating PLL Parameters from Unit Step Response

This section explains how to estimate the parameters of a second-order system or phase-locked loop (PLL) based on its response to a unit step input. This method is useful for validating filter performance or estimating internal settings from external observations.

Applicable System

The method applies to systems with the following transfer function:

$$H(s) = \frac{2\zeta\omega_n s + \omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$$

This represents the output of a PLL just after the integrator. It is the 'velocity-type' output of a second-order system that can track changing input related to speed.

Step Response Features to Measure

You can extract the following key features from the unit step response:

- Peak Value: Y_{max}
- Final Value: Y_{∞}
- Overshoot: $M_p = (Y_{max} - Y_{\infty})/Y_{\infty}$
- Time to Peak: T_p when the peak occurs

Formulas to Estimate Parameters

Estimate the damping factor (ζ) from overshoot:

$$\zeta \approx -\ln(M_p) / \sqrt{\pi^2 + \ln^2(M_p)}$$

Estimate the natural frequency (ω_n) from time to peak:

$$\omega_n \approx \pi / (T_p * \sqrt{1 - \zeta^2})$$

If needed, convert ω_n to frequency (Hz):

$$f_n = \omega_n / (2\pi)$$

Example

Measured response:

- Overshoot $\approx 16\%$
- Time to Peak ≈ 6 seconds

$$\rightarrow \zeta \approx 0.503$$

$$\rightarrow \omega_n \approx 0.53 \text{ rad/s}$$

$$\rightarrow f_n \approx 0.085 \text{ Hz}$$

Usage in Code

You can automate this analysis in VB.NET by:

- Collecting step response data
- Computing overshoot and time to peak
- Applying the above formulas

This is useful for validating PLL behavior in live systems or during development.

Generated by ChatGPT · OpenAI

How to design the PLL second order system based on the first order IIR standard response.

For the first order IIR filter we use this factor to define the term α :

$$\alpha = \frac{2}{N + 1}$$

This is the value of α that will give similar statistics for average and sigma equivalent to a pure FIR filter with a square data window of N points.

The transfer function $H(s)$ is defined as:

$$H(s) = \frac{1}{\tau s + 1} = \frac{\alpha}{s + \alpha}$$

$$\text{With } \alpha = \frac{1}{\tau}$$

For that single pole filter the 3 dB bandwidth is also given by:

$$\omega_{c1} = \frac{1}{\tau} \text{ (rad/sec)}$$

The 3 dB bandwidth of that filter is defined as:

$$\omega_{c1} = -\log(1 - \alpha)$$

or

$$f_c = \frac{\log(1 - \alpha)}{2\pi}$$

Origin of the Term $\log(1 - \alpha)$ in First-Order IIR Filters:

For a discrete-time first-order IIR low-pass filter defined as:

$$y[n] = \alpha x[n] + (1 - \alpha)y[n - 1]$$

The transfer function in the Z-domain is:

$$H(z) = \frac{\alpha}{1 - (1 - \alpha)z^{-1}}$$

Substituting $z = e^{j\omega}$ to analyze the frequency response:

$$H(e^{j\omega}) = \frac{\alpha}{1 - (1 - \alpha)e^{-j\omega}}$$

The -3 dB cutoff frequency ω_{c1} occurs where the magnitude drops to $\frac{1}{\sqrt{2}}$ of the DC gain. Solving the magnitude equation:

$$\left| \frac{\alpha}{1 - (1 - \alpha)e^{-j\omega}} \right| = \frac{1}{\sqrt{2}}$$

Leads to:

$$\omega_{c1} = -\log(1 - \alpha)$$

This logarithmic relationship arises because the pole of the filter is at $z = 1 - \alpha$, and the frequency response depends on how close this pole is to the unit circle in the Z-plane. The term ***log***(1 - α) expresses the -3 dB bandwidth in radians/sample based on this pole location.

Digital Frequency Ω vs. Physical Frequency in IIR Filters

In digital signal processing, we use:

$$z = e^{j\Omega}$$

Where:

- z is the complex frequency variable in the Z-transform
- Ω is the **digital angular frequency**, measured in **radians per sample**

Relationship Between Digital and Physical Frequency:

$$\Omega = 2\pi \cdot \frac{f}{f_s} \quad (\text{radians/sample})$$

$$f = \frac{\Omega \cdot f_s}{2\pi} \quad (\text{Hz})$$

Where:

- Ω = digital frequency (angular)
- f = physical (analog) frequency in Hz
- f_s = sampling rate in Hz

Frequency Range in Digital Systems:

- $\Omega = 0$: DC (0 Hz)
- $\Omega = \pi$: Nyquist frequency (half of the sampling rate)
- $\Omega \in [0, \pi]$ for single-sided spectrum
- $\Omega \in [-\pi, \pi]$ for full spectrum

π

Matching Cutoff Frequencies Across Filter Types

When designing filters of different orders (e.g., first-order and second-order), it is important to compare their cutoff frequencies **in the same domain**.

Digital Filters:

If both filters are implemented digitally (using the Z-transform), compare their -3 dB points using **digital angular frequency** Ω . This ensures they exhibit equivalent attenuation characteristics at the same point in the digital spectrum.

Analog Filters:

If you're comparing analog prototypes (in the Laplace domain), use analog angular frequency ω in $s = j\omega$. This applies before any conversion to digital.

Mixed Domain Designs:

If you design in the analog domain and then convert to digital (e.g., using the bilinear transformation), keep in mind that:

$$\Omega = 2 \cdot \tan^{-1} \left(\frac{\omega}{2f_s} \right) \quad \text{or} \quad \omega = 2f_s \cdot \tan \left(\frac{\Omega}{2} \right)$$

This non-linear relationship means:

- Equal analog cutoff frequencies do **not** map directly to equal digital cutoff frequencies
- Matching must account for **frequency warping** introduced by the transformation

Summary Table:

Goal	Compare In	Frequency Type
Analog design	$s = j\omega$	Analog frequency
Digital design	$z = e^{j\Omega}$	Digital frequency

Goal	Compare In	Frequency Type
Mixed design (analog and digital). Use bilinear transform Frequency warping applies		

Comparison: First-Order IIR vs. Second-Order DPLL Cutoff Alignment

Matching Cutoff Frequencies Across Filter Types

When designing filters of different orders (e.g., first-order and second-order), it is important to compare their cutoff frequencies in the same domain.

Digital Filters:

Use digital angular frequency Ω to compare -3 dB points.

Analog Filters:

Use analog angular frequency ω in $s = j\omega$.

Mixed Domain Designs:

$$\Omega = 2 \cdot \arctan(\omega/2f_s)$$

$$\omega = 2f_s \cdot \tan(\Omega/2)$$

Matching must account for frequency warping due to the non-linear transform.

Summary Table:

Goal	Compare In	Frequency Type
Analog design	$s = j\omega$	Analog frequency
Digital design	$z = e^{j\Omega}$	Digital frequency
Mixed design (analog → digital)	Use bilinear transform	Frequency warping applies

Comparison: First-Order IIR vs. Second-Order DPLL Cutoff Alignment

First-Order IIR Filter:

$$H_1(z) = \frac{\alpha}{(1 - (1 - \alpha)z^{-1})}$$

For that single pole filter the 3 dB bandwidth is also given by:

$$\Omega_c = \frac{1}{\tau} \text{ (rad/sec)}$$

The 3 dB bandwidth of that filter is defined as:

$$\Omega_{c1} = -\log(1 - \alpha)$$

or

$$\Omega_{c1} = \log\left(\frac{N+1}{N-1}\right)$$

$$\Omega_{c1} \approx -\log(1 - \alpha)$$

Second-Order DPLL Loop Filter:

$$H_2(z) = K \frac{(1 - z^{-1})^2}{(1 - 2rz^{-1} + r^2z^{-2})}$$

This equation can also be represented as the paper by Shayan and Lee-Ngoc “All digital PLL: Concepts, design and applications”

$$H(Z) = \frac{C_2(Z - 1) + C_1}{(Z - 1)^2 + C_2(Z - 1) + C_1}$$

With C_1 and C_2 are related to the S frequency domain ($j\omega$) of a second order analog PLL filter

$$H(S) = \frac{2\zeta\omega_n S + \omega_n^2}{S^2 + 2\zeta\omega_n S + \omega_n^2}$$

Where we have:

$$C_2 = 2\zeta\omega_n T_s = 2\zeta\Omega_s$$

$$C_1 = \frac{C_2^2}{4\zeta^2}$$

Solve for Ω where:

$$|H_1(e^{j\Omega})| = |H_2(e^{j\Omega})| = 1/\sqrt{2}$$

Conclusion:

- Use Ω to compare frequency behavior in digital systems.
- Second-order filters offer steeper roll-off but must be aligned in bandwidth.

Comparison: First-Order IIR vs. Second-Order DPLL Cutoff Alignment

When comparing a first-order IIR filter with a second-order DPLL-based loop filter, it is crucial to define the target -3 dB cutoff frequency in terms of digital frequency Ω to ensure consistent behavior.

First-Order IIR Filter:

A standard one-pole low-pass IIR filter is defined by:

$$H_1(z) = \frac{\alpha}{(1 - (1 - \alpha)z^{-1})} =$$

The pole is at $z = 1 - \alpha$, and the -3 dB cutoff occurs approximately at:

$$\Omega_{c1} = -\log(1 - \alpha)$$

with

$$\alpha = \frac{2}{N + 1}$$

The equation becomes:

$$\Omega_{c1} = \log\left(\frac{N + 1}{N - 1}\right)$$

The term N here is the number of points that the user needs for data filtering. The relation ensure that the statistics at the output of the filter will be equivalent to the one of an

equivalent FIR first order filter using a square window of N points. This establishes the statistical relation to the equivalent FIR filter.

This gives also a smooth single-pole roll-off at -20 dB/decade.

And another relation that is needed is the equivalence in bandwidth between the IIR filter and the second order IIR DPLL filter. The number of point N for the second order filter should be scaled or adjusted to ensure the 3 dB bandwidth equivalence. The statistic between the first order and second order may diverge slightly but the difference will become mainly related to the order of the filter and the sharper roll-off filtering of -40 dB/decade.

Second-Order DPLL Loop Filter:

As defined in the work of Shayan and Le-Ngoc ("All-Digital PLL: Concepts, Design and Applications"), the second-order digital loop filter has the transfer function:

$$H_2(Z) = \frac{C^2(Z-1)+C^1}{(Z-1)^2+C^2(Z-1)+C^1}$$

Where C_1 and C_2 are also related to the analog S-domain transfer function:

$$H_2(S) = \frac{2\zeta\omega_n S + \omega_n^2}{S^2 + 2\zeta\omega_n S + \omega_n^2}$$

.

.

defined as:

$$C_2 = 2\zeta\omega_n T_s = 2\zeta\Omega_n 1$$

$$C_1 = C_2^2 / (4\zeta^2)$$

To match the bandwidths of the first and second order filters, solve for Ω where:

$$|H_1(e^{j\Omega})| = |H_2(e^{j\Omega})| = 1/\sqrt{2}$$

Additionally, the 3 dB bandwidth for the second-order analog PLL filter in the analog domain can be approximated by:

$$\omega_{c2} \cong 2(\zeta + 1/4\zeta)\omega_n$$

This 3 dB cut-off frequency can be mapped to the digital filter equivalent Ω_{c2} using the bilinear transformation. We then have:

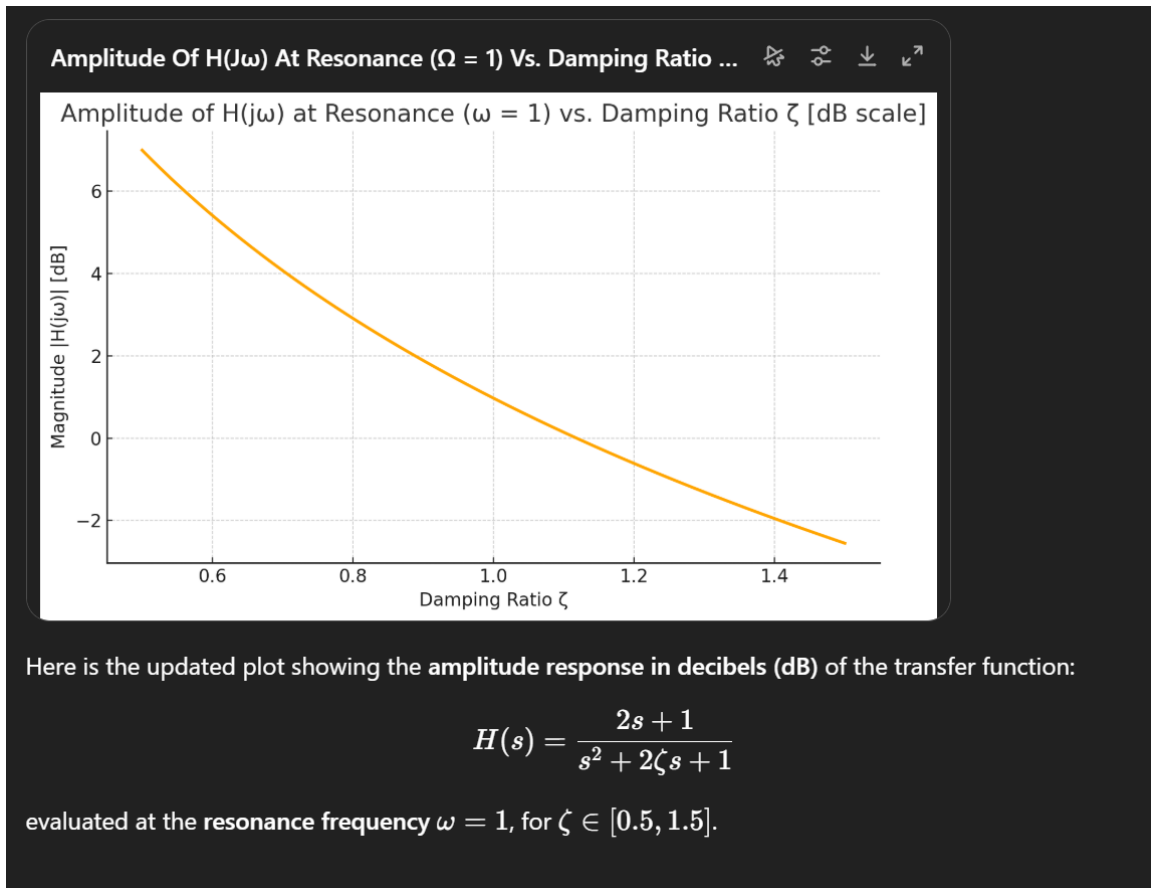
Or

$$\Omega_{c2} = 2 \cdot \arctan(\omega_{c2}/2f_s)$$

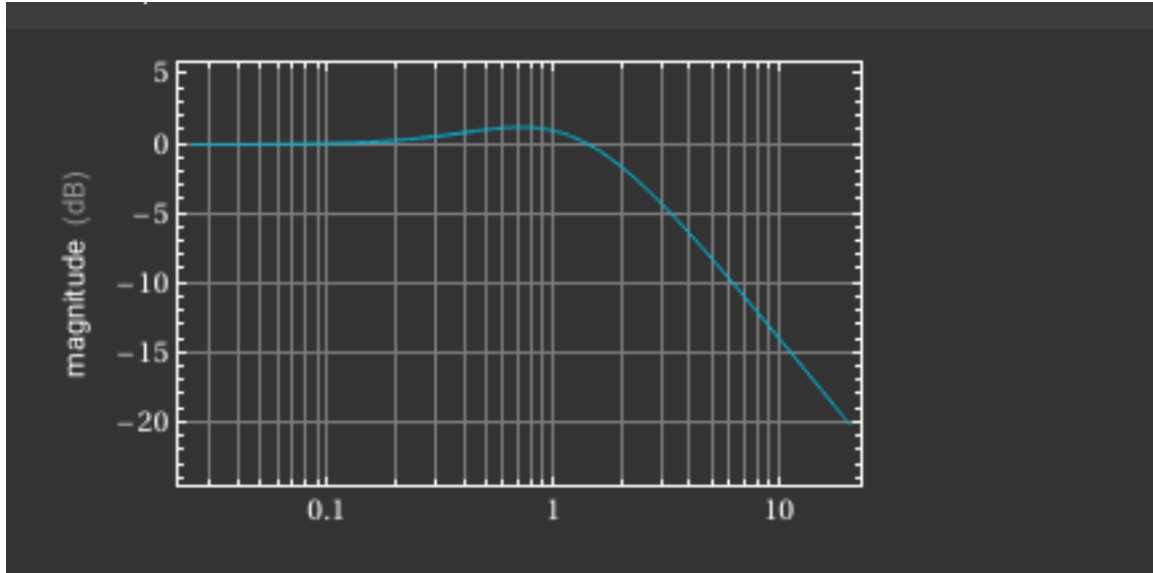
and by substitution it follows:

$$\Omega_{c2} \cong 2 \cdot \arctan((\zeta + 1/4\zeta)\Omega_n)$$

As a first approximation the amplitude of the analog second order filter at the resonance frequency ω_n is always very close to unity even with a damping factor variation of between 0.7 to 1.2. This is demonstrated in the following graph



Next is the frequency response for the same filter for a damping factor of 1



In the current application the DPLL filter is expected to be used mostly as a fast roll off low pass filter with the main advantage being its fast response and relatively small delay. Under this consideration it appears to be reasonable to assume that the resonance frequency of the second order filter should match the 3 dB bandwidth of the first order filter i.e.

From the discussion above for the first order filter we have:

$$\Omega_{c1} = -\log(1 - \alpha)$$

with

$$\alpha = \frac{2}{N + 1}$$

Again, the value for α here is to ensure statistical equivalence to a square windows FIR filter. After some transformation we the equation for the digital frequency becomes:

$$\Omega_{c1} = \log\left(\frac{N + 1}{N - 1}\right)$$

Since we need $\Omega_{n2} = \Omega_{c1}$ we can write the following relation between the first and second order filter:

$$\Omega_{n2} = \Omega_{c1} = \log\left(\frac{N + 1}{N - 1}\right)$$

Using the linear transformation this yield to the 3 dB bandwidth of the ADPLL defined as:

$$\Omega_{c2} \cong 2 \cdot \arctan((\zeta + 1/4\zeta)\Omega_{n2})$$

From this we can evaluate the parameters C2 in function of N as:

$$C_2 = 2\zeta\omega_n T_s = 2\zeta\Omega_{n2} = 2\zeta \log\left(\frac{N+1}{N-1}\right)$$

Taylor Series Expansion of $\log((N+1)/(N-1))$

We aim to find the Taylor series expansion of the logarithmic expression:

$$\log((N+1)/(N-1))$$

This can be rewritten as:

$$\log((1+1/N)/(1-1/N))$$

Using the known identity for the logarithm of a ratio:

$$\log((1+x)/(1-x)) = 2(x + x^3/3 + x^5/5 + \dots), \text{ for } |x| < 1$$

Set $x = 1/N$, which is valid for $N > 1$. Then:

$$\log((N+1)/(N-1)) = 2(1/N + 1/(3N^3) + 1/(5N^5) + \dots)$$

Therefore, the Taylor series expansion is:

$$\log((N+1)/(N-1)) = 2/N + 2/(3N^3) + 2/(5N^5) + \dots$$

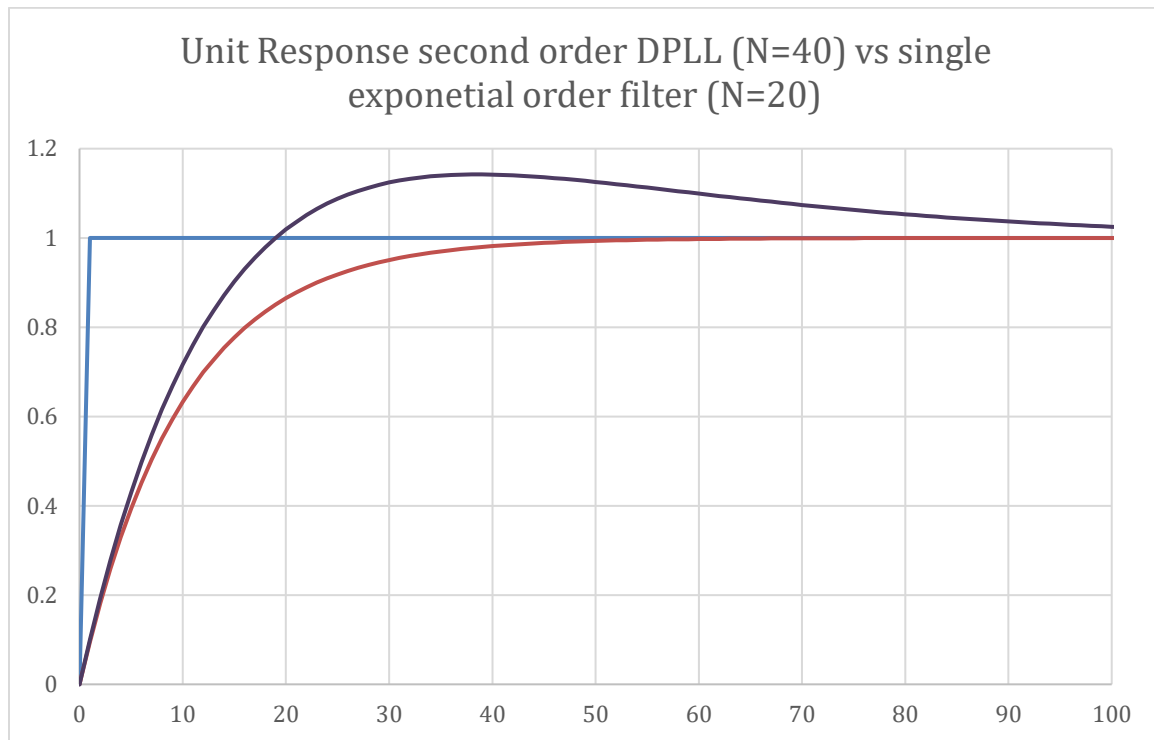
This expansion becomes very accurate for large N. and the equation above for C2 could be written as:

$$C_2 = 2\zeta\omega_n T_s = 2\zeta\Omega_{n2} = 2\zeta \log\left(\frac{N+1}{N-1}\right) \cong 2\zeta\left(\frac{2}{N}\right)$$

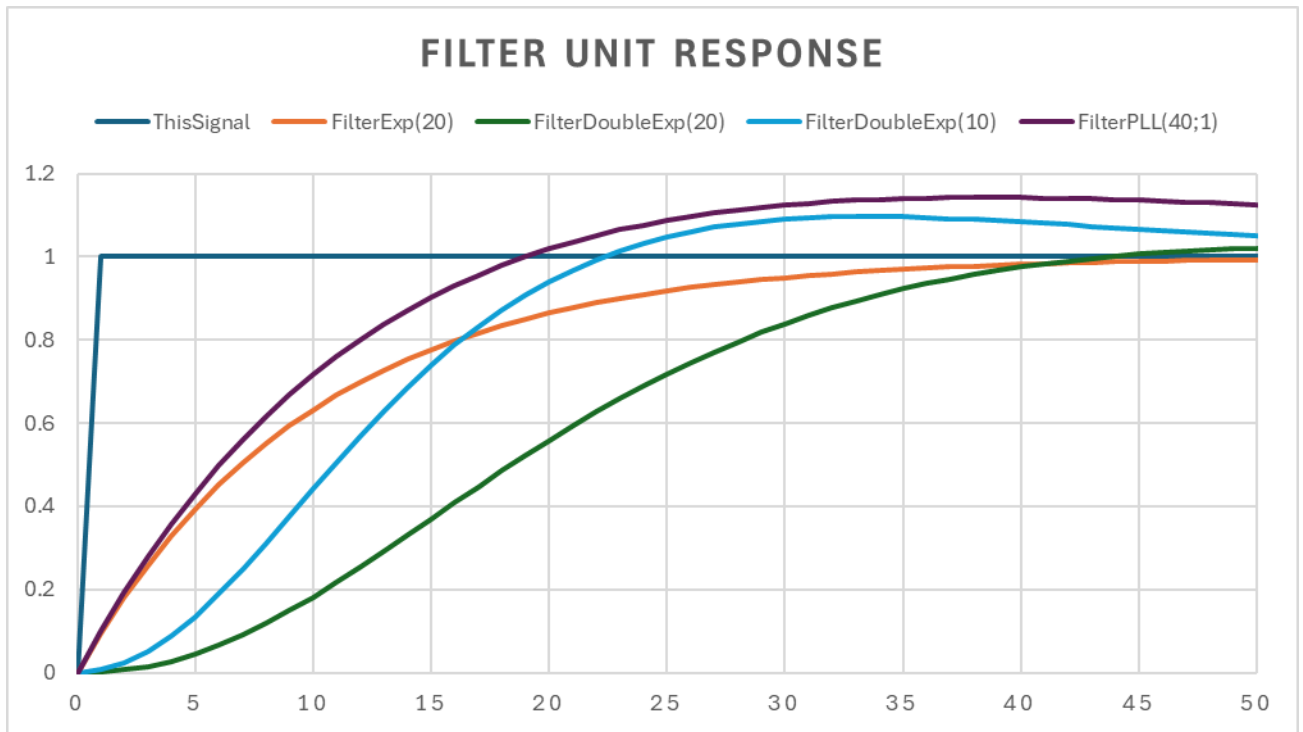
The previous filter code used the approximation factor $2/N$ for estimating the factor C_2 but new code has been updated to reflect the exact equation formula for the factor C2. The effect for the filter response is minimal but now matches the exact theoretical formula

equivalence between the 3 dB bandwidth of the IIR filter and the resonance frequency of the second order filter.

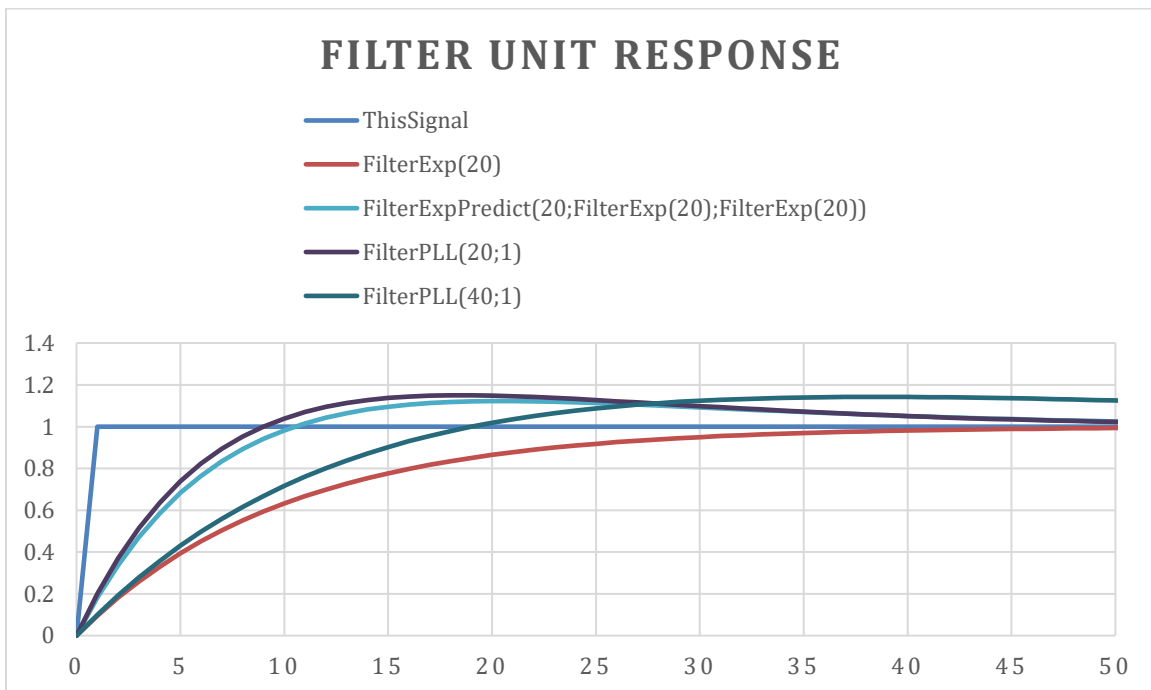
Note that the implication is that the corresponding 3 dB bandwidth for the second order filter is larger than the equivalent IIR first order filter of the current implementation. In case the frequency bandwidth at 3 dB is needed, it could be achieved approximatively by doubling the number of samples parameters N for the DPLL filter. The result for a unit response time will become very similar to the single pole filter as the dominant factor for the pulse response is the bandwidth of the filter. The difference is the expected overshoot of the DPLL and the response of the unit signal becomes very similar as demonstrated by the following response of the two filters:



Another aspect to consider is the double filter exponential build by cascading two exponential first order filter with the same 3 dB bandwidth or parameters. The unit response is shown in the figure below compared to the single IIR filter with $N=20$ for all filter except the DPLL set a $2N$. The result shows the slower response that can be improved by reducing the factor N by factor 2 as shown below. We then have an equivalent second order filter with slightly slow response speed but reduced overshoot.:

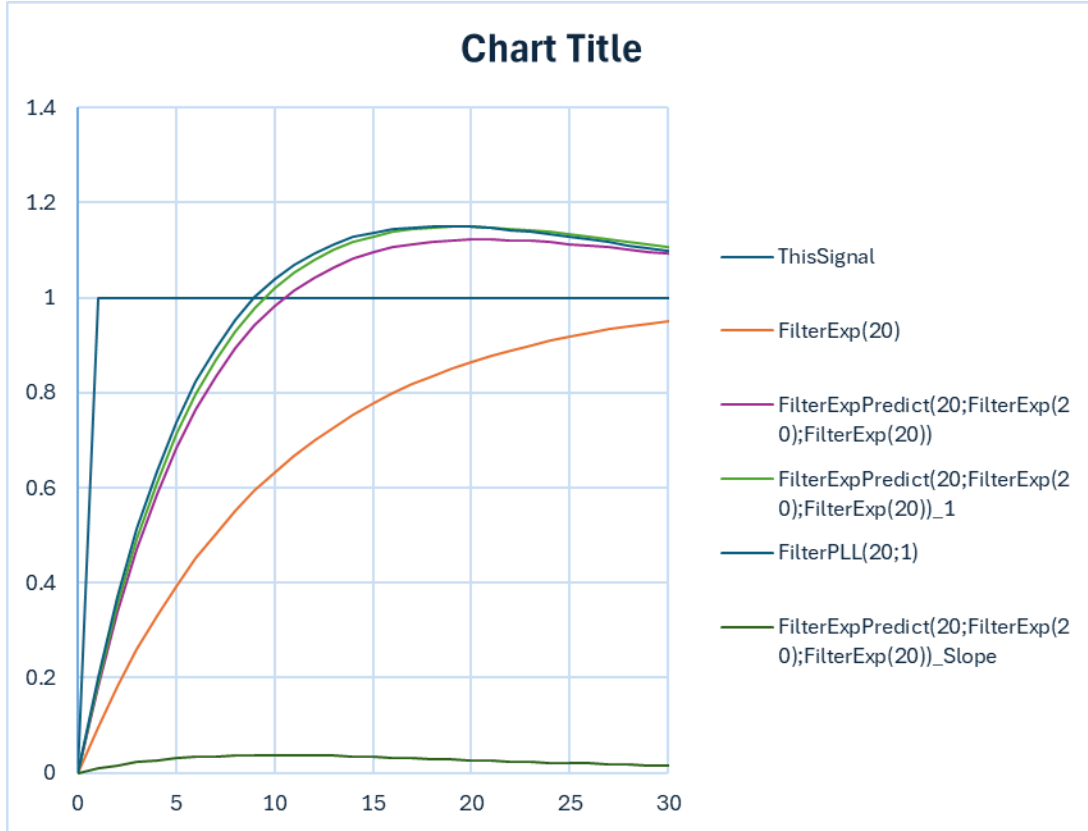


The other filter of interest is the FilterExpPredict based on the Brown prediction model of two IIR first order exponential filter. In this case we have a response that is very similar to the equivalent second order DPLL filter as shown below:

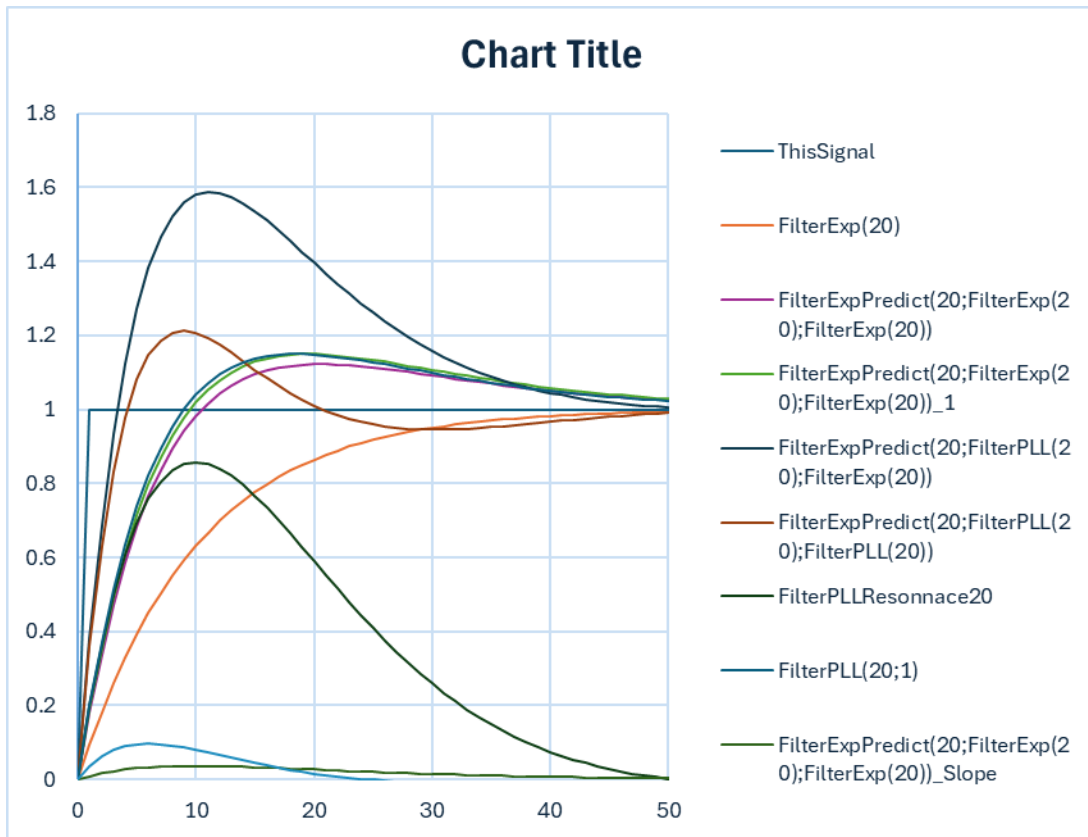


Next on this filter an interesting aspect is that the Brown exponential filter with a prediction of 1 (see line orange vs the brown line is essentially and in all practical cases identical to the DPLL response. In practice, filter could be used with practically the same result i.e. :

$$FilterPLL(N = 20, \zeta = 1) \cong FilterExpPredict(N = 20, k_{prediction} = 1)$$



Some other more unusual filter are possible if one combine the various filter with the Brown FilterExp standard filter are shown here



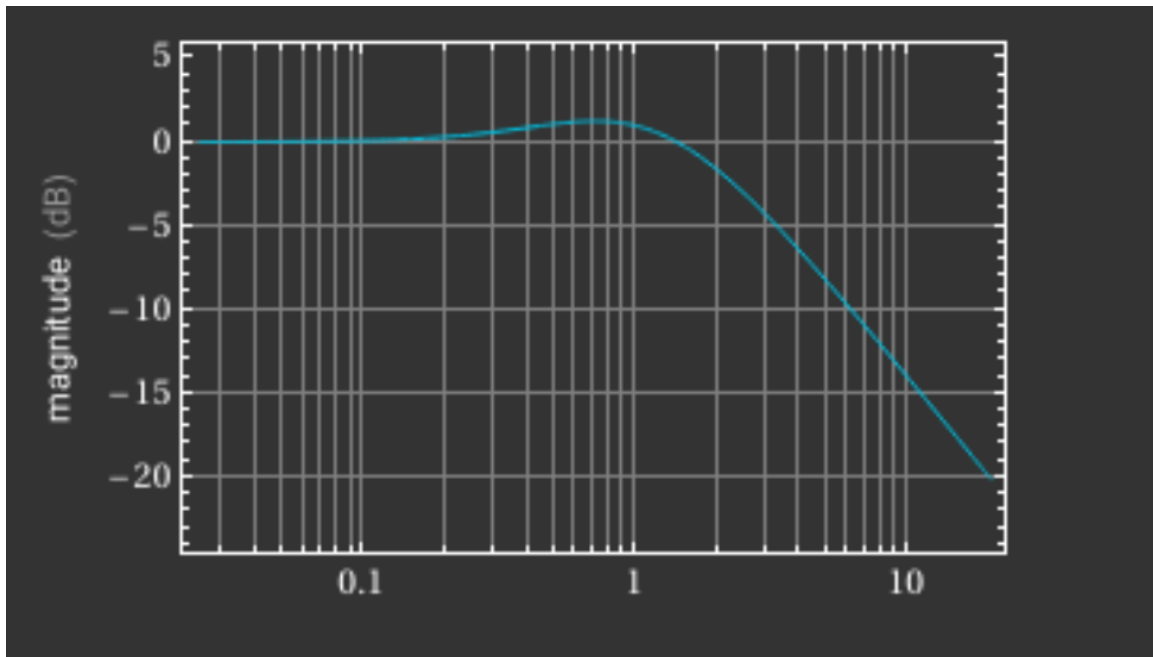
Amplitude Response for FilterExpPredict

This filter amplitude is close in response to the PLL second order but with a smaller overshoot, and an attenuation at frequency 10 of -14dB.

Input interpretation

systems model

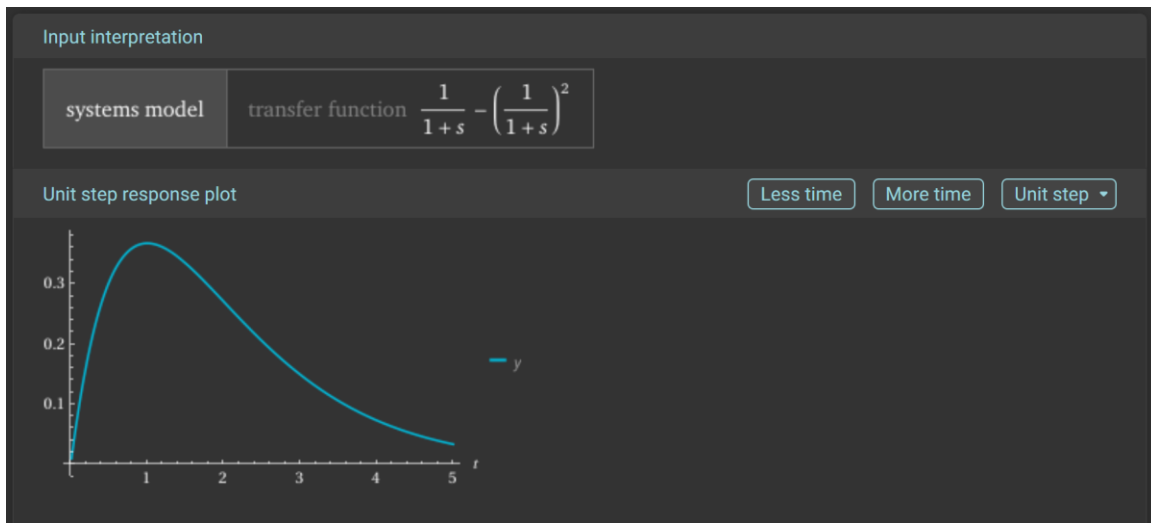
transfer function $\frac{2}{1+s} - \frac{1}{1+s} \times \frac{1}{1+s}$



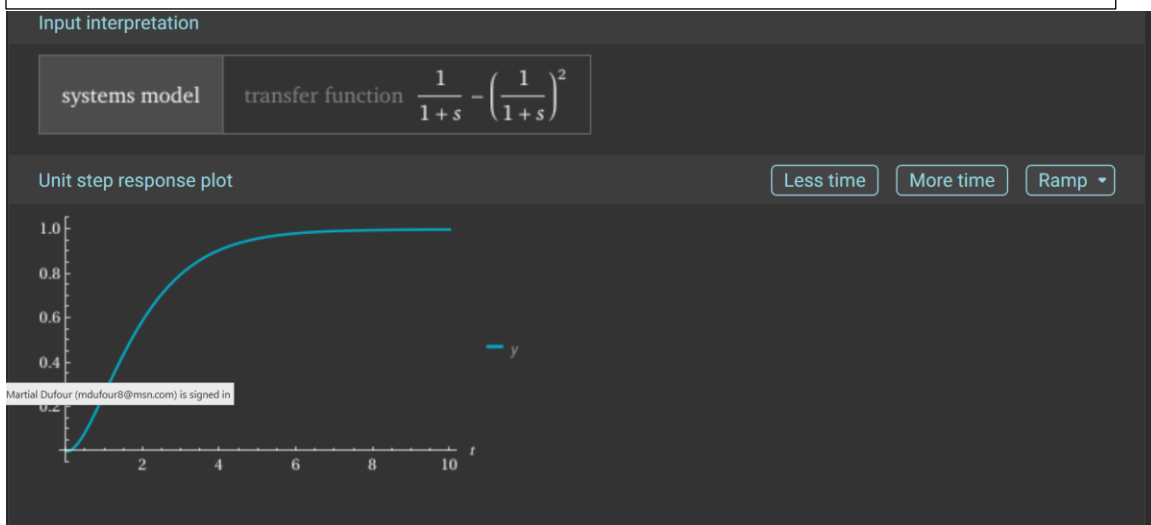
This filter is easily modified to extract the trend of the filtering process. The Trend amplitude response for FilterExpPredict is given below. Note the -20 dB drop at frequency 0.1 and 10. However there is an attenuation at the frequency 1 of -8.5 dB and if taken as a reference the drop at frequency 10 is effectively only of -11.5. The trend represents an estimate of the slope of the filter center around frequency 1 essentially acting as a bandpass filter operating with an approximative loss of -8.5 dB at the center frequency.

In a digital filter the trend output of the filter is also adjusted for calibration in unit/sample by the factor $2/(N-1)$. The filter output then represents the best slope estimate that can be used if needed for extrapolation to the next samples. When the trend is used the signal output becomes very similar to the result that would be obtained using a second order DPLL operating with a damping factor of 1. As demonstrated below in the FilterPLL section

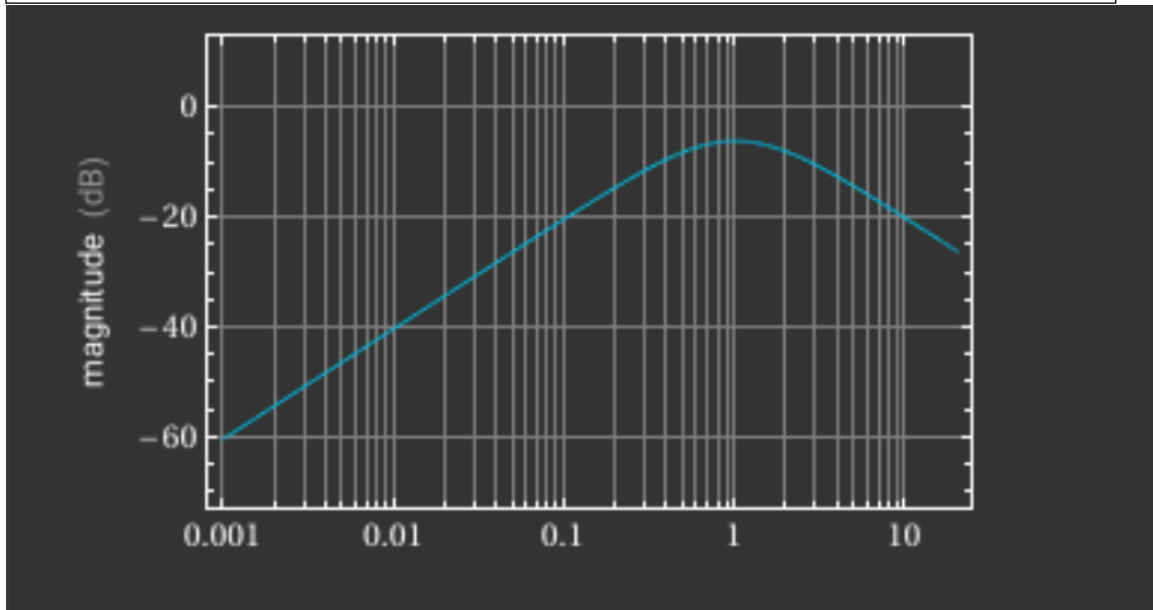
FilterExpPrediction slope response to a Unit Step



FilterExpPrediction slope response to a Ramp Step (converge toward 1)



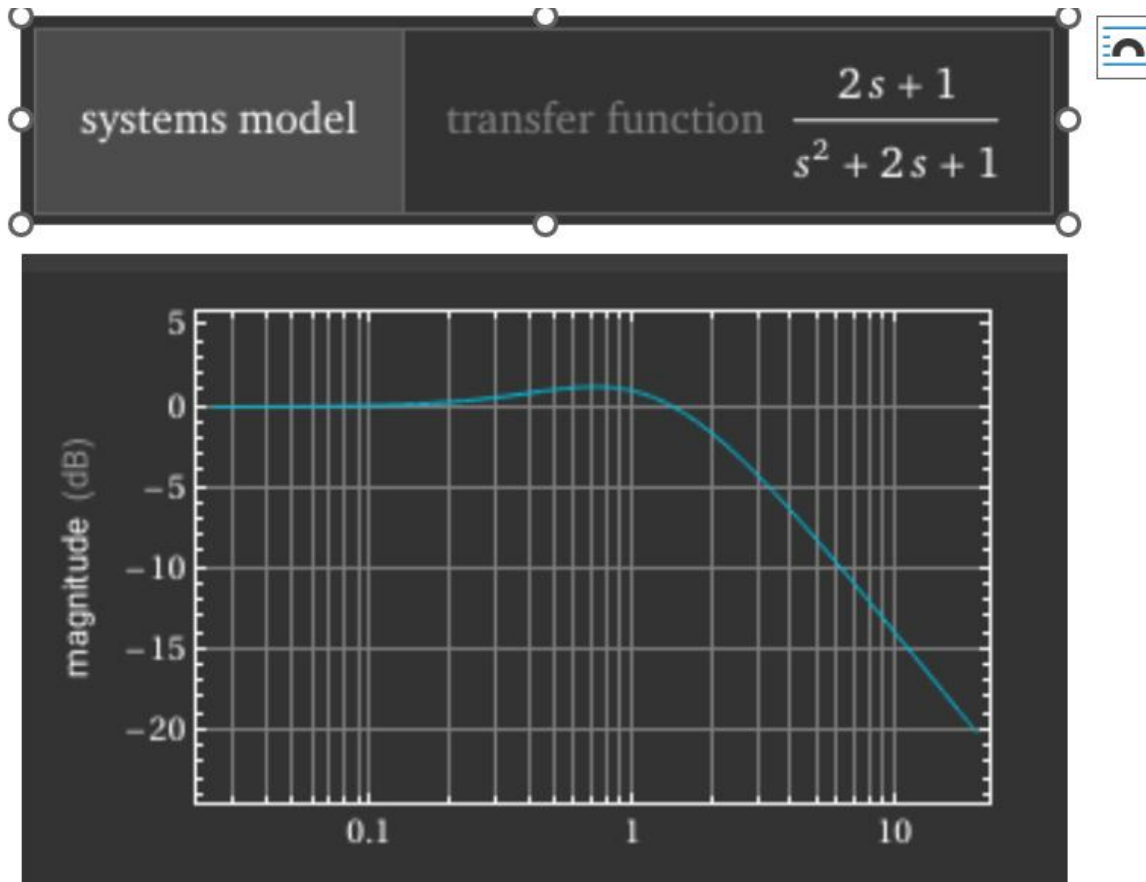
FilterExpPrediction Frequency Response



Note that if we test this system with a Unit ramp the long-term response tends toward 1 i.e. the slope per sample is one:

Amplitude Response for the FilterPLL

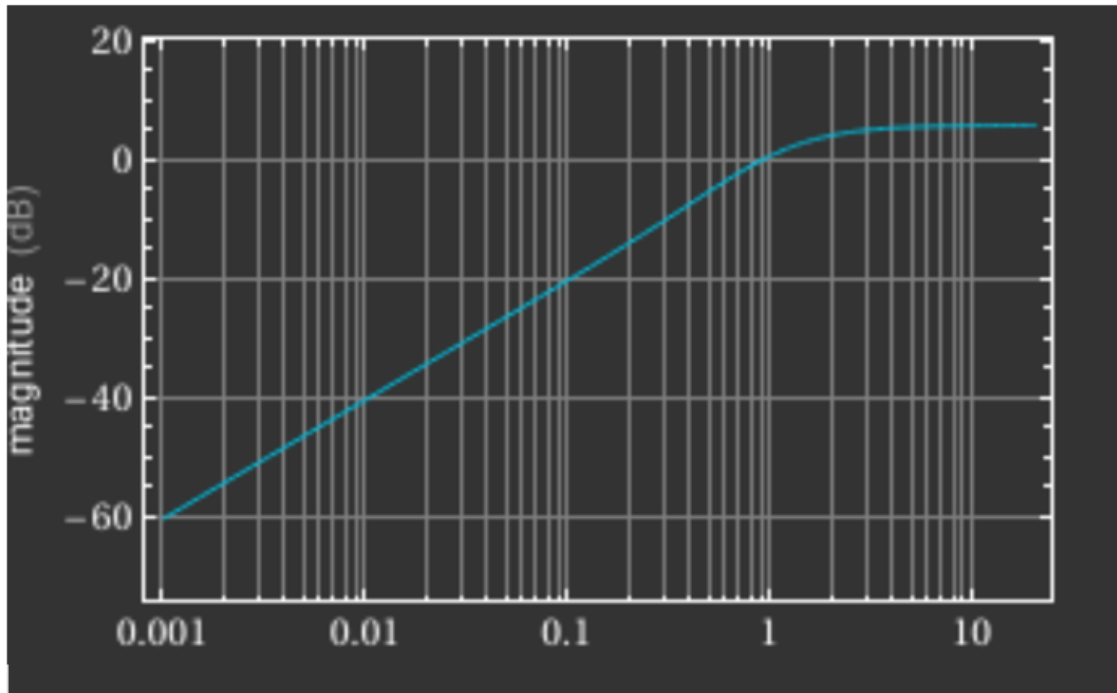
The following graph represents the response of a standard PLL with a resonance frequency and damping factor respectively of 1. We observe a slight overshoot of approximately 15% and a drop at frequency 10 of about 14.5 dB similar the brown exponential filter presented above.



This filter derivative is also easily obtained if the signal is taken just before the input to the integrator. In that case the frequency response becomes:

systems model

$$\text{transfer function } s \times \frac{2s + 1}{s^2 + 2s + 1}$$



As expected, this filter becomes a high pass filter with a high frequency response of +6 db. In most cases this filter can be use in a very similar way that the Brown filter if the output of the input to the integrator is filtered with a double exponential filter as indicated below. We note a very sinusoidal shaped pulse response with a slight undershoot. The High frequency attenuation is large with a factor of -28 db relative to the center frequency of the filter. The rejection a the low frequency is not as goo at -14dB relative to the center frequency. This design can be vey good for high frequency rejection.

The ramp test indicates that this system can also be used to measure the slope with accuracy as indicated below with the long-term response showing a slope of one as expected. The stabilization period, however, appear to be slightly better than the brown filter with a stabilization at around 4-5 samples versus 5-6 for the brown filter.

Input interpretation

systems model

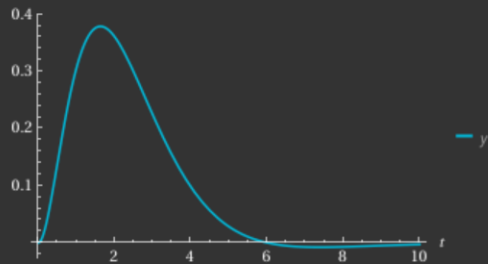
transfer function $\left(s \times \frac{2s+1}{s^2+2s+1} \right) \times \frac{1}{(1+s)^2}$

Unit step response plot

Less time

More time

Unit step ▾



Input interpretation

systems model

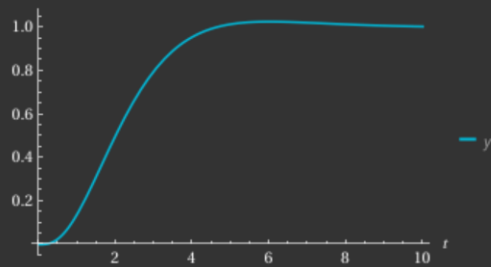
transfer function $\left(s \times \frac{2s+1}{s^2+2s+1} \right) \times \frac{1}{(1+s)^2}$

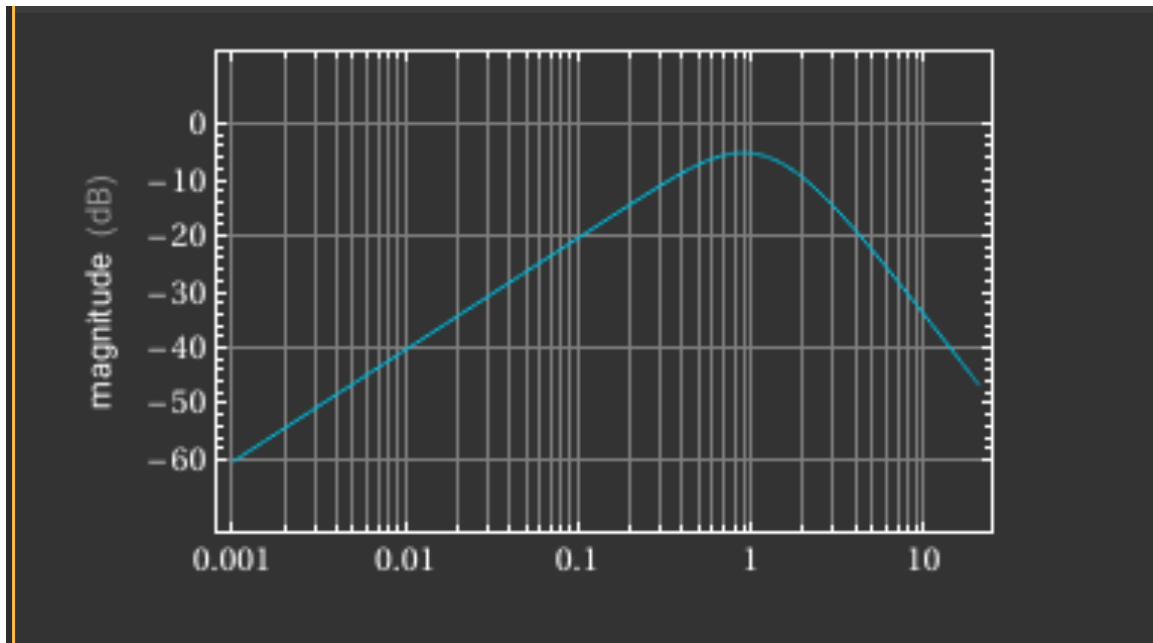
Unit step response plot

Less time

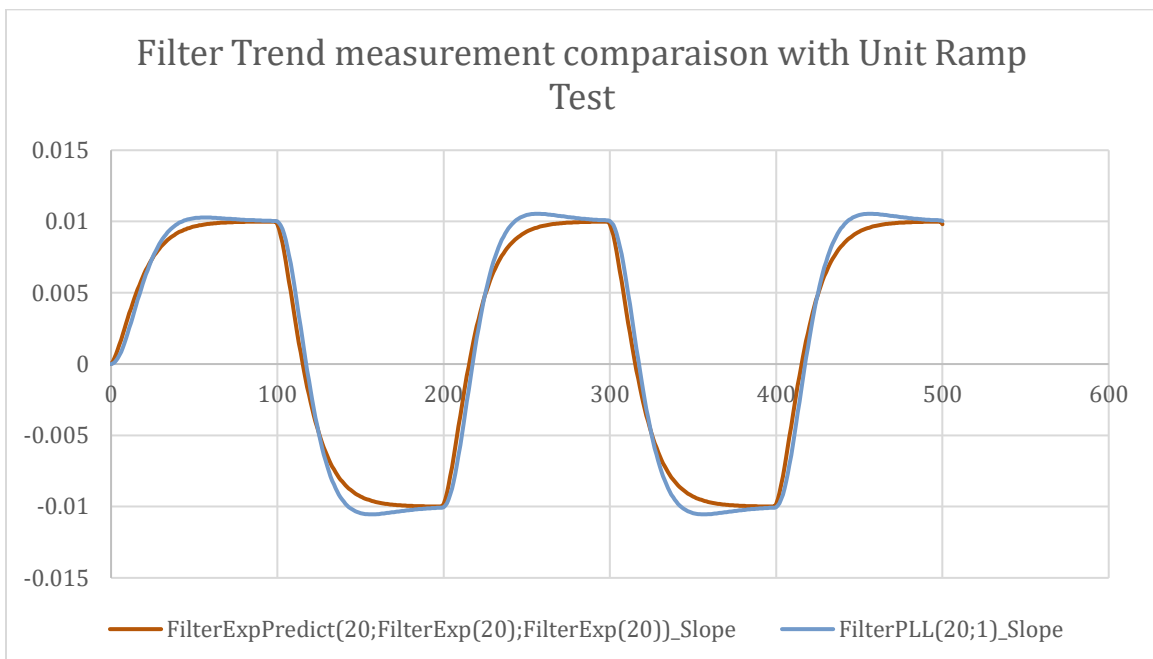
More time

Ramp ▾





The result obtained when we compare the DFPP response to the Brown exponential filter when a ramp is applied to the input is essentially very close with as expected a slight overshoot for the DPLL response.



The resonance PLL filter

A variation of the PLL filter is slightly different than the brown exponential filter can be obtained if only the derivative portion of the PLL response is kept in the function. The filter can be implemented by subtracting the PLL response using a double exponential filter. The result of the transfer function of that filter is typically of that form:

$$H(s) = \frac{2s + 1}{s^2 + 2s + 1} - \frac{1}{(1 + s)^2}$$

This filter is easy to implement, and the formula is easily matched and exact if the damping factor is defined as 1. However, even for a damping factor of between 0.7 and 1.5 the result is still acceptable in an application where a perfect signal response is not an absolute requirement. The aspect to consider is that the slope estimate for this filter should be divided by 2 for an exact slope measurement. The following graph presents the different results:

systems model

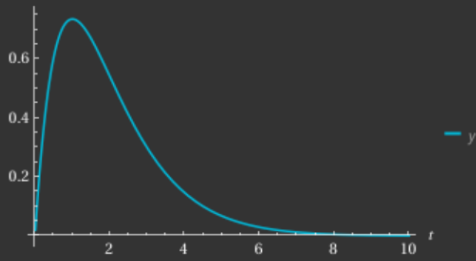
$$\text{transfer function } \frac{2s + 1}{s^2 + 2s + 1} - \frac{1}{(1 + s)^2}$$

Unit step response plot

Less time

More time

Unit step ▾



State space representation

$$\dot{x} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -1 & -4 & -6 & -4 \end{pmatrix} x + \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} u$$

$$y = (0 \ 2 \ 4 \ 2)x + 0u$$

(unique up to a similarity transformation)

Properties

number of inputs/outputs: 1/1 (SISO system) | order: 4

Transfer function element zeros

$s = -1$ (order 2 zero)

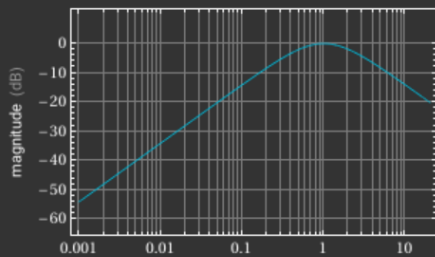
$s = 0$ (simple zero)

Transfer function element poles

$s = -1$ (order 2 pole)

Bode plot

Show stability margins





Bandwidth Analysis for the Filter $H(s) = ks / (s^2 + ks + 1)$

Transfer Function:

$$H(s) = ks / (s^2 + ks + 1)$$

This is a second-order pass pass filter.

Objective:

Determine the 3 dB bandwidth as a function of the parameter k .

Step 1: Magnitude Response

We compute the magnitude of the transfer function:

$$|H(j\omega)| = |jk\omega / ((j\omega)^2 + k(j\omega) + 1)| = k\omega / \sqrt{((1 - \omega^2)^2 + (k\omega)^2)}$$

We seek ω_{3dB} where:

$$|H(j\omega_{3dB})| = (1/\sqrt{2}) * |H(j\omega)|_{\text{peak}}$$

However, this is difficult to solve algebraically. Instead, we approximate using the standard second-order form.

Step 2: Identify Standard Second-Order Form

Compare:

$$s^2 + ks + 1 \quad \text{with} \quad s^2 + 2\zeta\omega_n s + \omega_n^2$$

From this we identify:

- $\omega_n = 1$
- $2\zeta\omega_n = k \Rightarrow \zeta = k / 2$

Step 3: Approximate Bandwidth Formula

For an underdamped second-order system, the -3 dB bandwidth is approximated by:

$$\omega_{BW} \approx \omega_n \sqrt{(1 - 2\zeta^2 + \sqrt{(4\zeta^4 - 4\zeta^2 + 2)})}$$

Substitute $\zeta = k / 2$, $\omega_n = 1$:

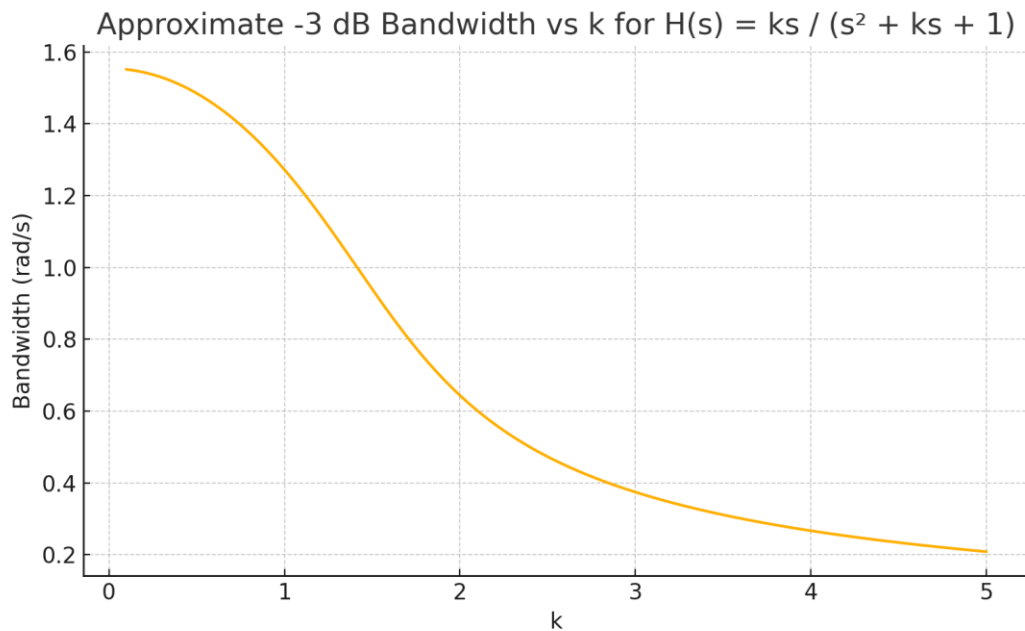
$$\omega_{BW}(k) \approx \sqrt{(1 - k^2/2 + \sqrt{(k^4/4 - k^2 + 2)})}$$

Step 4: Numerical Examples

k	ζ	Bandwidth (rad/s)
0.5	0.25	1.32
1.0	0.5	1.10
2.0	1.0	0.64
3.0	1.5	0.44

Step 5: Graphical Representation

The graph below shows how the bandwidth decreases as k increases:



This behavior reflects that increasing k increases damping, which reduces the bandwidth.

Conclusion:

The filter $H(s) = ks / (s^2 + ks + 1)$ has a decreasing -3 dB bandwidth as k increases, due to its increasing damping effect.

The formula derived above provides a practical way to estimate the bandwidth for different values of k .

Theory related to the Laplace Transform and the method used to estimate the initial and final value of a transform $H(s)$

Initial and Final Value Analysis using Laplace Transform

This document demonstrates how to compute the initial slope and final value of a system's response using Laplace Transform. These techniques are useful in control systems and signal analysis to determine system behavior at the beginning and end of the response.

System Description

Given Transfer Function:

$$H(s) = (2s + 1) / (s^2 + 2s + 1)$$

Input: Unit Step Function

$$U(s) = 1 / s$$

Output in Laplace Domain

$$Y(s) = H(s) \times U(s) = (2s + 1) / [s \times (s^2 + 2s + 1)]$$

Initial Slope (dy/dt at $t = 0^+$)

Using the Initial Derivative Theorem:

$$dy(t)/dt|_{t=0^+} = \lim_{s \rightarrow \infty} [s^2 \times Y(s)] = \lim_{s \rightarrow \infty} [s^2(2s + 1) / (s \times (s^2 + 2s + 1))]$$

Simplifying:

$$\text{Numerator: } s^2(2s + 1) = 2s^3 + s^2$$

$$\text{Denominator: } s(s^2 + 2s + 1) = s^3 + 2s^2 + s$$

As $s \rightarrow \infty$, divide numerator and denominator by s^3 :

$$\lim_{s \rightarrow \infty} [(2 + 1/s) / (1 + 2/s + 1/s^2)] = 2$$

→ Initial slope at $t = 0^+$ is **2**.

Final Value ($y(t)$ as $t \rightarrow \infty$)

Using the Final Value Theorem:

$$y(\infty) = \lim_{s \rightarrow 0} [s \times Y(s)] = \lim_{s \rightarrow 0} [s \times (2s + 1) / (s \times (s^2 + 2s + 1))]$$

Canceling s :

$$\lim_{s \rightarrow 0} (2s + 1) / (s^2 + 2s + 1) = 1 / 1 = 1$$

→ Final value as $t \rightarrow \infty$ is **1**.

Summary

- Initial Slope at $t = 0^+$: 2
- Final Value as $t \rightarrow \infty$: 1

Discrete-Time (Z-domain) Equivalent Concepts

In the Z-domain, equivalent theorems exist for analyzing the initial and final behavior of discrete-time systems. These theorems mirror the Laplace-domain analysis but are adapted for sampled systems.

Final Value Theorem (Z-domain)

For a stable causal system with output $Y(z)$ and input $X(z)$, the final value of $y[n]$ is given by:

$$\lim_{n \rightarrow \infty} y[n] = \lim_{z \rightarrow 1} (1 - z^{-1}) Y(z)$$

This is analogous to taking the limit as $s \rightarrow 0$ in the Laplace domain. It assumes all poles of $(1 - z^{-1})Y(z)$ lie inside the unit circle, except possibly a simple pole at $z = 1$.

Initial Value Theorem (Z-domain)

The initial value of $y[n]$ (i.e., $y[0]$) is given by:

$$\lim_{n \rightarrow 0^+} y[n] = \lim_{z \rightarrow \infty} Y(z)$$

This matches the intuition that early behavior is governed by the high-frequency (large z) characteristics.

About $z = -1$ (Nyquist Frequency)

The point $z = -1$ corresponds to $e^{j\pi}$, i.e., the Nyquist frequency in discrete-time systems. This point is important in frequency response analysis but is not used for determining time-domain behaviors such as initial value, final value, or slope. Therefore, it is not applicable to initial/final value theorems.

Discrete-Time Approximation of Slope

There is no direct analogue in the Z-domain for computing the initial slope as in Laplace. Instead, the slope at the start can be approximated using finite differences:

$$dy[n]/dt \approx (y[1] - y[0]) / T_s$$

Where T_s is the sampling period. This gives an estimate of the rate of change at the beginning of the signal.

Analysis of Final Slope Behavior

While initial and final values can be directly extracted from Laplace and Z-domain transforms, the slope at the final value (i.e., the derivative as time approaches infinity) cannot be directly extracted.

Laplace Domain Perspective

The Final Value Theorem provides:

$$\lim_{t \rightarrow \infty} y(t) = \lim_{s \rightarrow 0} s \times Y(s)$$

To compute the slope at $t \rightarrow \infty$ (i.e., dy/dt as $t \rightarrow \infty$), one might consider:

$$\lim_{s \rightarrow 0} s^2 \times Y(s)$$

However, this expression actually yields the initial slope (as $t \rightarrow 0^+$), not the final slope. Laplace transforms emphasize initial behavior due to the high-frequency weighting of s .

Z-Domain Perspective

Similarly, in the Z-domain, initial and final values are computed using limits at $z \rightarrow \infty$ and $z \rightarrow 1$ respectively. There is no direct method to extract the final slope (i.e., the difference $y[n] - y[n-1]$ as $n \rightarrow \infty$) from $Y(z)$.

Interpretation for Stable Systems

For most stable systems, the output $y(t)$ (or $y[n]$) converges to a steady value as time increases. This implies that the slope (rate of change) at large time approaches zero:

$$\lim_{t \rightarrow \infty} dy(t)/dt = 0$$

This is typically the case for causal, asymptotically stable systems. If a system exhibits exponential decay like $y(t) \approx A \cdot e^{(-\alpha t)}$ near the end, then the final slope also decays exponentially to zero.

Conclusion on the use of the Laplace and Z transform to obtain the initial and final value analysis.

While Laplace and Z-transforms provide valuable tools for analyzing system dynamics, extracting the slope at the final time directly from these domains is not possible. Such information typically requires knowing the explicit time-domain form of $y(t)$ or its long-term decay behavior.

Other consideration related to the use of the Laplace transform:

We often use Laplace Transform for evaluating the response of an analog system equivalent which corresponds in realities to an infinite sampled system. In practice we are dealing with a sampled system often better described via the Z transform. The Z transform takes into account the limit imposed by the sampling. When we use the s transform in a sampled system the goal is to use the known properties of certain filter in the analog world to understand the system and then if possible adapt the digital system in an equivalent often very similar response via the Z transform.

In all this document the s transform is often used to extract key information and concept and then the Z transform relation is used to extract the final relation for any filter. An example is the use of the standard second order analog PLL response to understand the quasi-equivalent comportment of the digital system and for this we use:

$$H(s) = (2s + 1) / (s^2 + 2s + 1)$$

Based on the analysis done in other section of this document with the filter above we have from the corresponding relation to the Z transform, assuming $\zeta=1$ we can write Ω_{n2} as:

$$\Omega_{n2} = \Omega_{c1} = \log \left(\frac{N + 1}{N - 1} \right)$$

This function $H(s)$ above corresponds to a digital resonance frequency in the z domain of $\Omega_{n2} = 1$. In that situation we can extract the value of N and show that it is defined as:

$$N = \frac{(1 + e)}{(1 - e)} = \coth \left(\frac{1}{2} \right) \cong 2.164$$

Or environ 2 samples. It is important to realize that the property derived from this $H(s)$ are still valid in the z transform or the samples system but only if we used a larger sampling rate normally greater than 5 or the DPLL become unstable. The current code detects the stability range and automatically gives an error if the parameters N is too small. It is recommended to use at least $N=5$ as the minimum filter rate for the DPLL filter.

+-

Conclusion:

- For comparing different filter structures in a digital system, digital frequency Ω is the correct axis of comparison
- Use established PLL formulas (e.g., Shayan & Le-Ngoc) for second-order bandwidth estimation
- Solve numerically or graphically for frequency where both filter magnitudes reach $1/\sqrt{2}$ to ensure matched 3 dB performance

More detailed plots and step responses can further clarify their behavior, especially in time-domain tracking or stability analysis.